

DIGITAL SIGNAL PROCESSING

Project 1

Full Name: Nikos Katsaidonis

Reg. Number: 0 3 1 2 1 8 6 8

Part 1 - Telephone Touch – Tones

●The purpose of this exercise is to **construct the telephone tones** and **locate the key** that has been pressed. In addition, we will become familiar with discrete **Fourier transform** (DFT) as well as **signal distortion** in the time domain.

Question 1.1

d0 = np.sin(0.7217*n) + np.sin(1.0247*n)

d1 = np.sin(0.5346*n) + np.sin(0.9273*n)

d2 = np.sin(0.5346*n) + np.sin(1.0247*n)

d3 = np.sin(0.5346*n) + np.sin(1.1328*n)

d4 = np.sin(0.5906*n) + np.sin(0.9273*n)

d5 = np.sin(0.5906*n) + np.sin(1.0247*n)

d6 = np.sin(0.5906*n) + np.sin(1.1328*n)

d7 = np.sin(0.6535*n) + np.sin(0.9273*n)

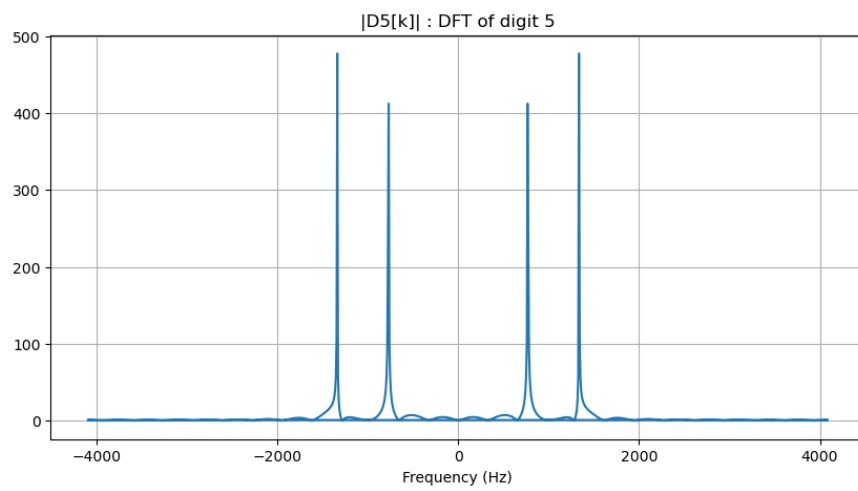
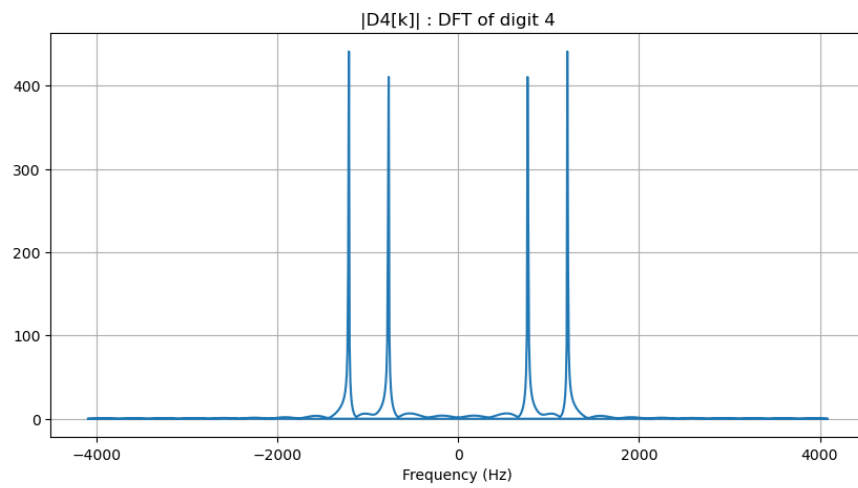
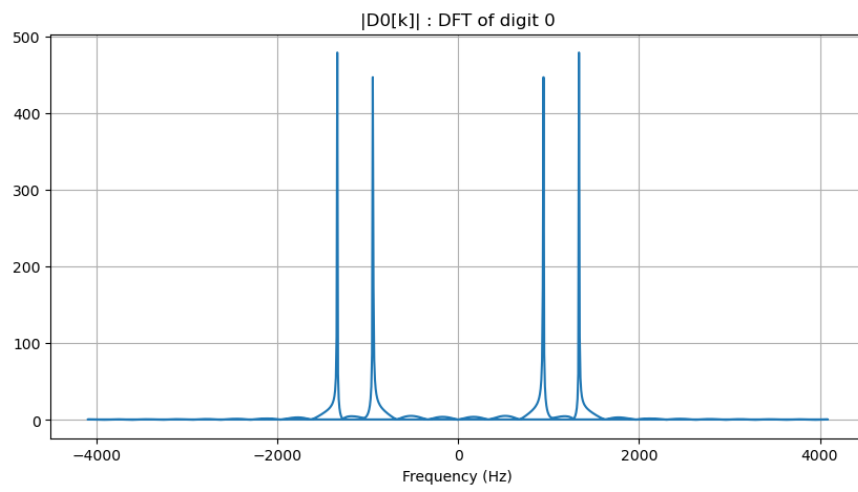
d8 = np.sin(0.6535*n) + np.sin(1.0247*n)

d9 = np.sin(0.6535*n) + np.sin(1.1328*n)

We construct the above signals corresponding to the tones and with the function **display(Audio())** (sampling frequency **fs=8192Hz**) we can listen to each of them.

Question 1.2 We calculate **DFT 1024** points of **0,4,5 tons** using the **fft.fft()** and **fft.fftfreq()** function of numpy.

Here is the result:



As expected, each DFT consists of **4 peaks** symmetrically in pairs due to the **two sine** present in each tone. The three graphs have quite close the frequencies at which they make their peaks, which is justified by the very close **00** to the initial signals.

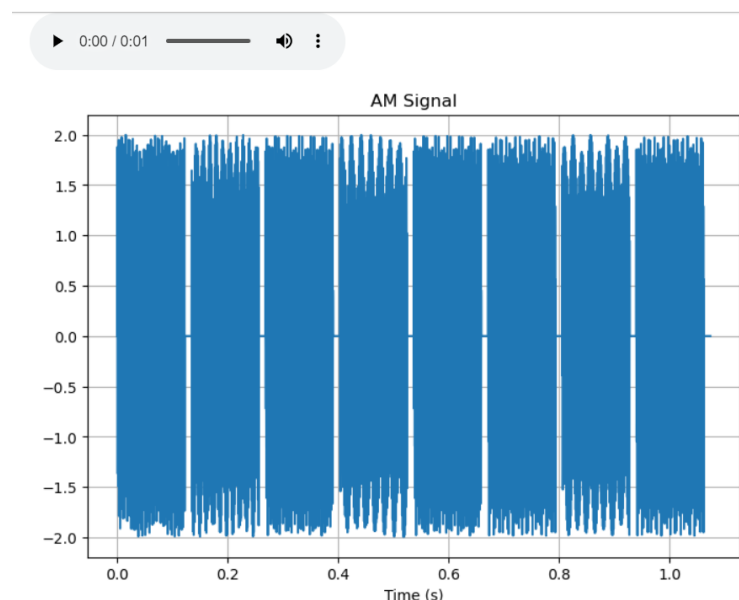
Question 1.3

Given the registration number **AM = 0 3 1 2 1 8 6 8**, we create a list of elements (**AM_signal_list**) containing the corresponding tones after **extending each of them with 100 zeros** with the help of the **concatenate()** function of numpy so that there is no difficulty distinguishing tones during listening and we create a new list (**extended_signals_list**). Using **the write()** of **scipy.io.wavfile** we write the result to the audio file "**tone_sequence.wav**" and make it **display(Audio())** in order to listen to it.

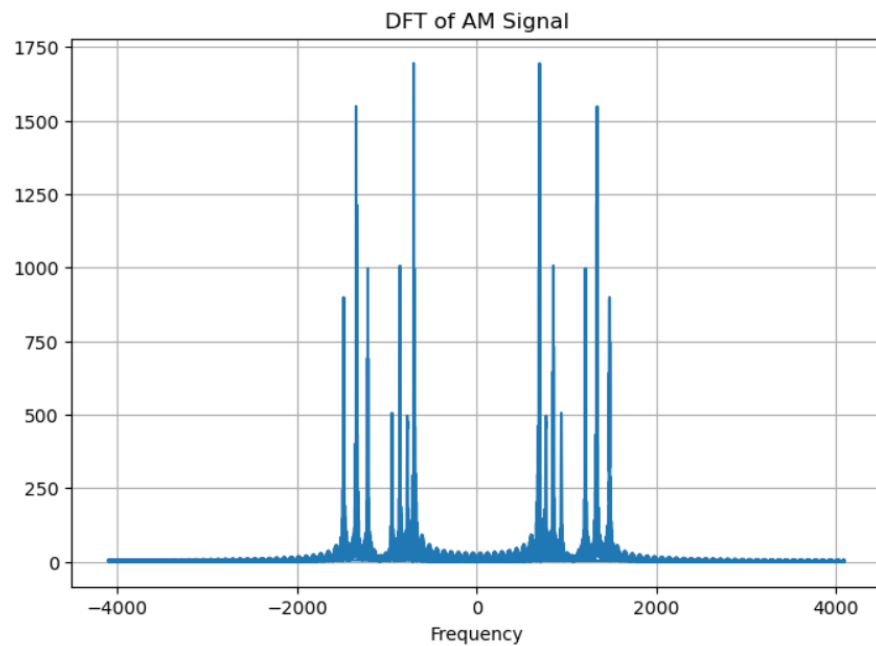
*Note that we encountered a problem writing to the file and added AM normalization : **normalized_AM = np.int16((AM/AM.max()) * 32767))**

****Also, the tone_sequence.wav appears to hear one less tone while the display(Audio()) sounds normal.**

We list the **signal in time** and its **DFT** below



Tonal signal AM = 0 3 1 2 1 8 6 8

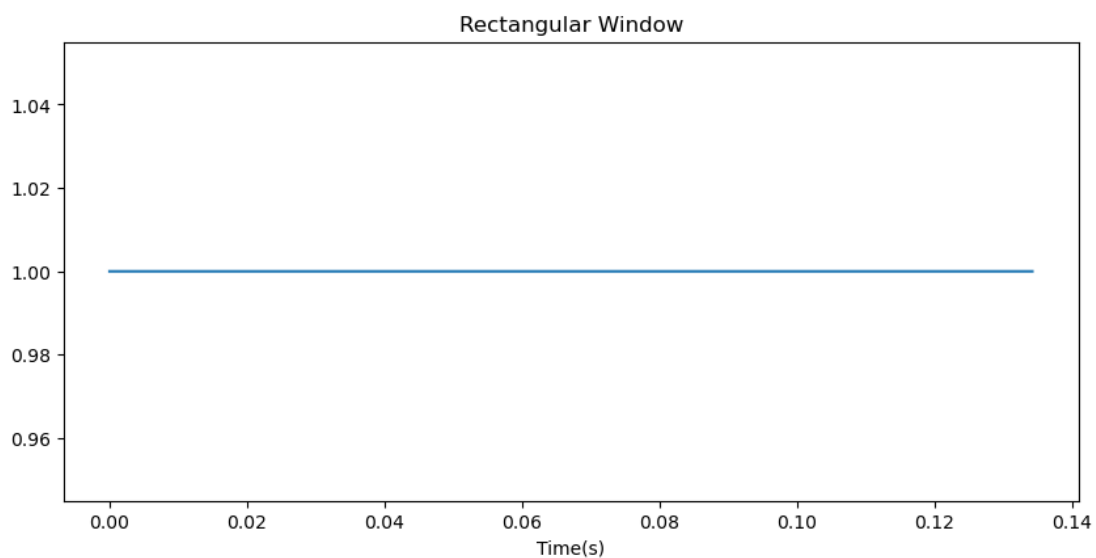


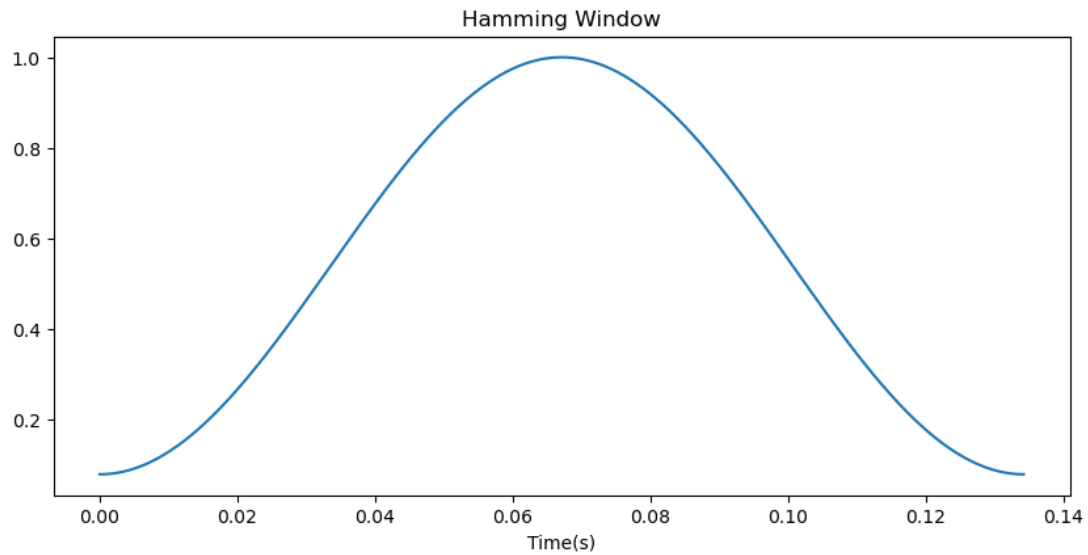
MT Tone Signal DFT = 0 3 1 2 1 8 6 8

We observe fewer peaks than expected as some digits are repeated in the signal (1.8) and therefore there is overlap of the peaks as well as because the peaks are very close to each other in the spectrum.

Question 1.4

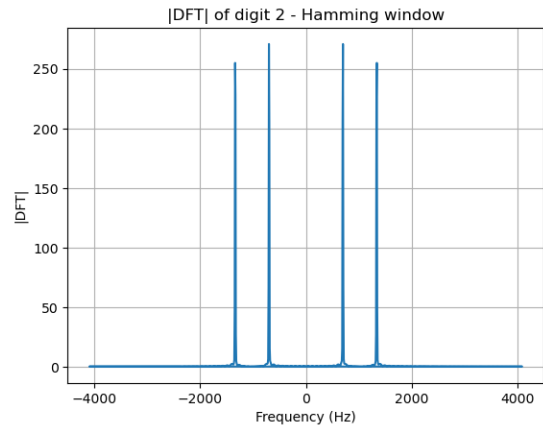
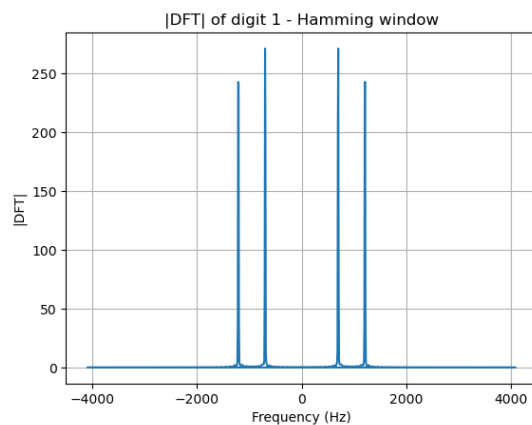
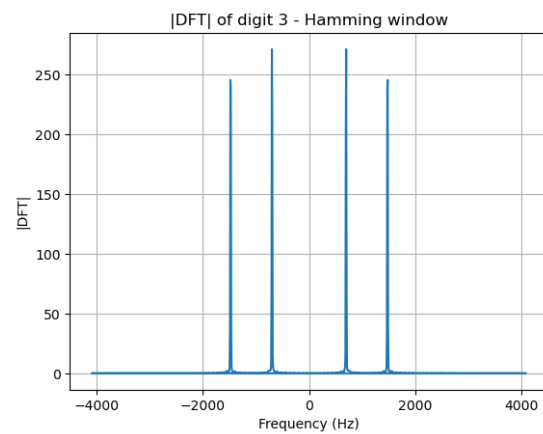
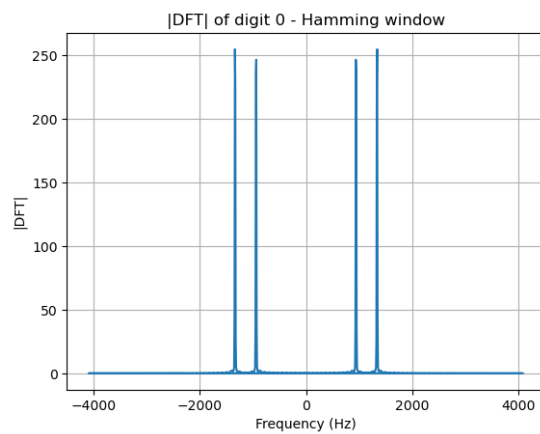
In this question we will create **hamming windows** and **square** length $L=1100$ i.e. as much as our tones after adding 100 zeros:

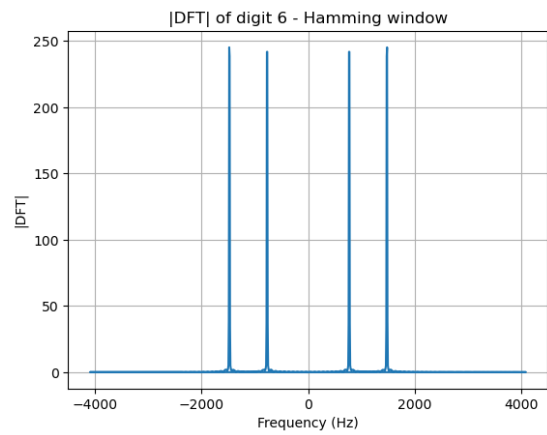
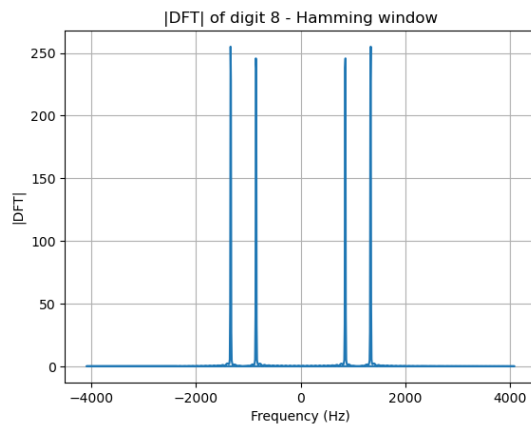




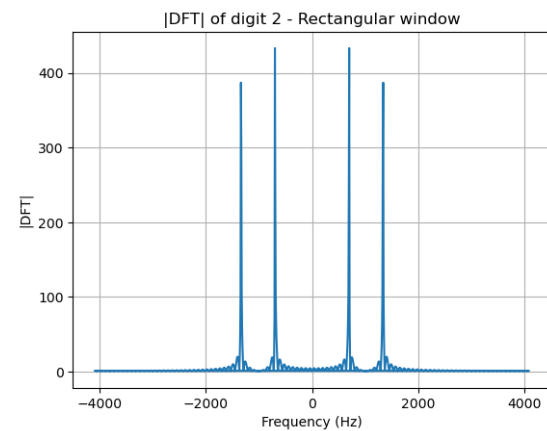
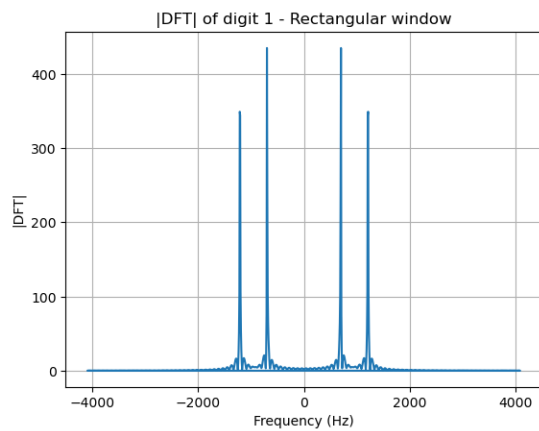
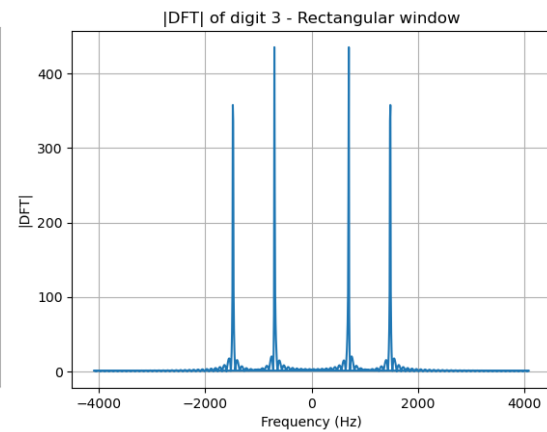
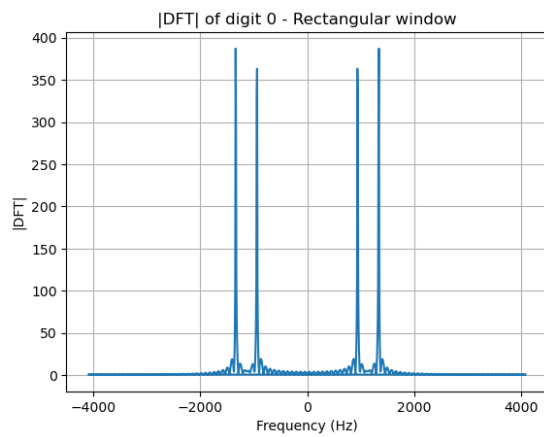
We window the tones [d0,d3,d1,d2,d1,d8,d6,d8] with the square and hamming windows and graphically represent the results (tones 1,8 are repeated 2 times in the AM signal):

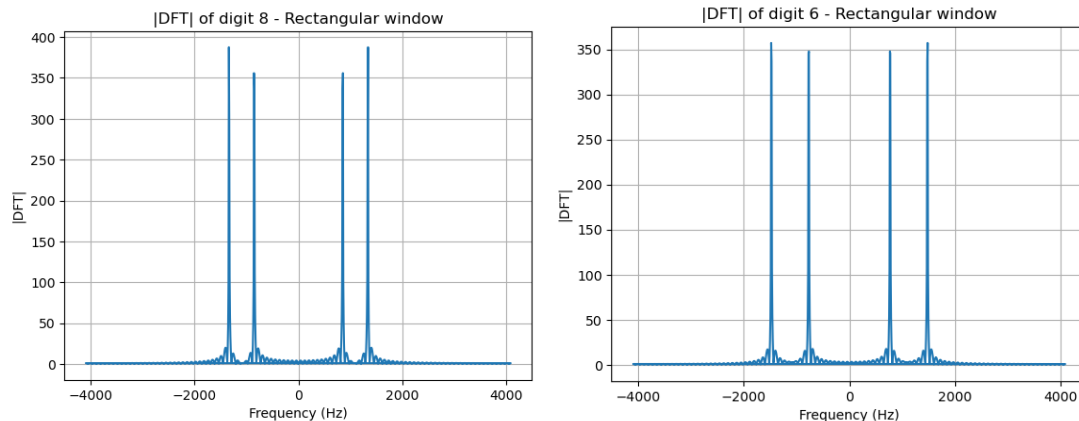
Windows Hamming:





Square windows:





What we notice when comparing the two windows is that with the **square window** we have a much **wider width of the central lobes**. In addition, we see **higher spectral leakage, which means that we disperse energy over a wider frequency range (see small peaks at low frequencies)**.

Question 1.5

Each sine signal, since it has specific angular frequency values, provides us with a DFT with peaks at specific points. After creating a list of all DFT tones (**DFTs_of_tones**), we calculate the energy of each DFT around each frequency (**from 0 to the length of DFT since DFT is symmetric to zero and therefore it is enough to find the positive frequencies**) and put it in a list (**energies**). Next, we run through the list containing the energies around each frequency (for each tone) and for the **largest and second highest energy of the tone, we store their frequency in the list k**. So now we have the frequencies of DFT in which each digit presents the two largest values of its energy. **These are characteristic of each digit**. We list the message printed as well as the pair of the two largest energies of each tone corresponding to the peaks.

List of k =

digit0:[167, 118]

digit1:[937, 87]

digit2:[167, 857]

digit3:[185, 87]

digit4:[151, 96]

digit5:[167, 857]

digit6:[185, 96]

digit7:[151, 873]

digit8:[167, 857]

digit9:[185, 917]

```
The two greater energies of digit 0
[229076.79430721106, 199350.37704250013]
The two greater energies of digit 1
[216220.8361380998, 216220.83613809978]
The two greater energies of digit 2
[227455.31999699745, 227455.3199969973]
The two greater energies of digit 3
[221957.65273139894, 216948.59833747908]
The two greater energies of digit 4
[194312.06347057933, 168323.35386232645]
The two greater energies of digit 5
[228014.76713314166, 228014.7671331416]
The two greater energies of digit 6
[221573.57742929985, 168704.09347711658]
The two greater energies of digit 7
[194296.27568024144, 194296.27568024138]
The two greater energies of digit 8
[227823.60072293025, 227823.6007229302]
The two greater energies of digit 9
[221808.9511575135, 150805.3735174227]
```

Question 1.6

The basic idea is that upon receiving the signal, the function calculates Fourier, detects which pair of frequencies from the k-list appears, and decodes the corresponding tone digits pressed.

Question 1.7

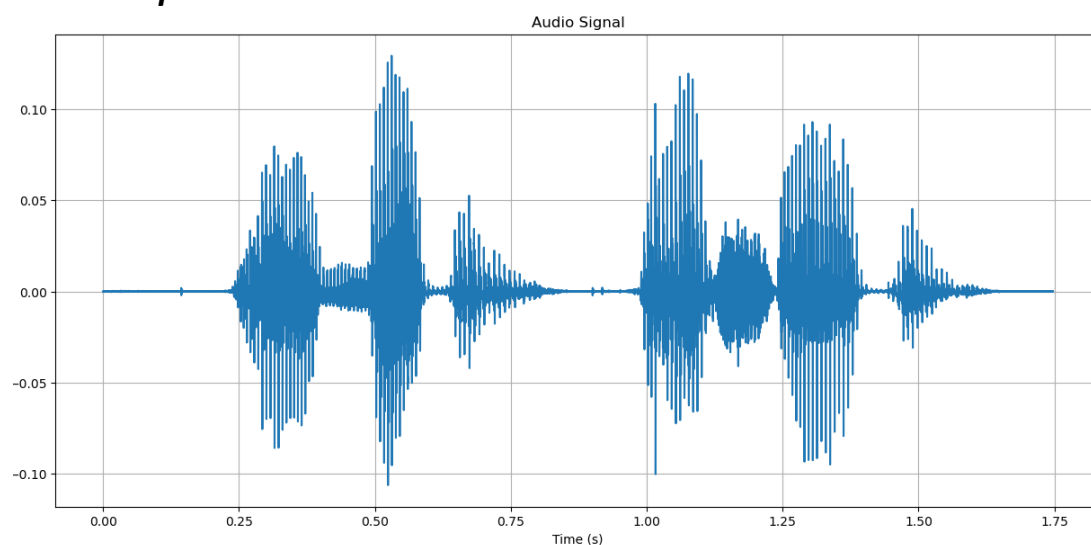
Sequential application of ttdecode to the marks "easySig", "ediumSig" and "hardSig" and display of their digits.

Part 2 – Short-Time Characteristics of Voice and Music Signals.

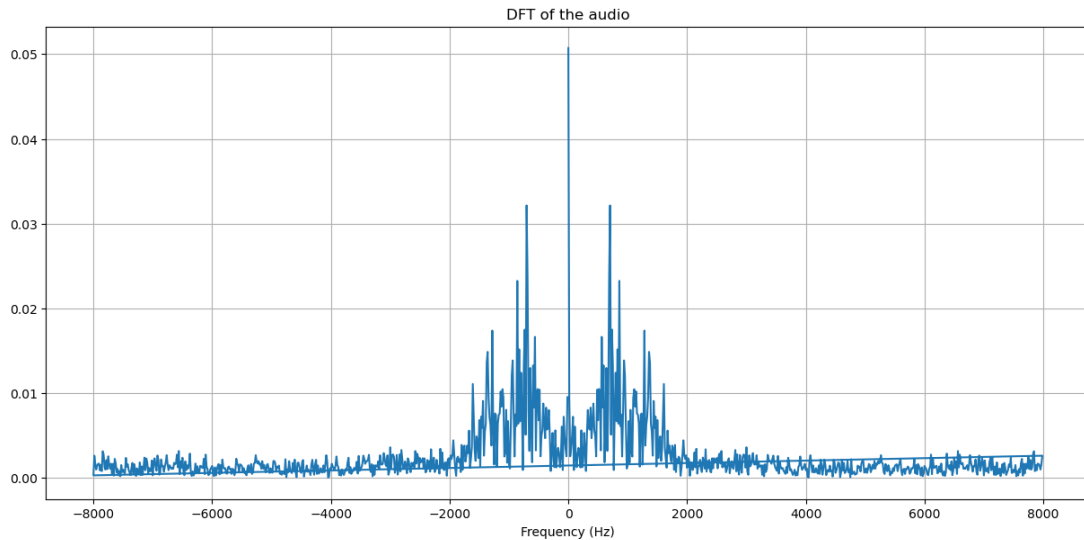
- The purpose of this exercise is to familiarize students with **short-term measurements**. These are measurements taken either in **the time domain or in the frequency field**, both with the help of a rolling window. **The first category includes the Short Time Energy and the Zero Crossing Rate of the signal**) while from the second category we will deal with the measurement of **the Spectral Centroid and the Spectral Flux**.

Question 2.1

In this question we will process the **voice signal speech_utterance.wav** which the complementary Project material is provided. Listening to it, we find that it is the message "Oh Mother, I saw them." We load the signal with the **librosa load()** function with a sampling frequency equal to **$f_s = 16\text{kHz}$** . We display the **1.75 seconds long** signal in time as well as the **1024-point** DFT of the trade mark. The results are shown below:



Voice signal in the time domain



DFT 1024 voice signal points.

We observe that the signal spectrum extends between the frequencies - **8kHz** – **8kHz**.

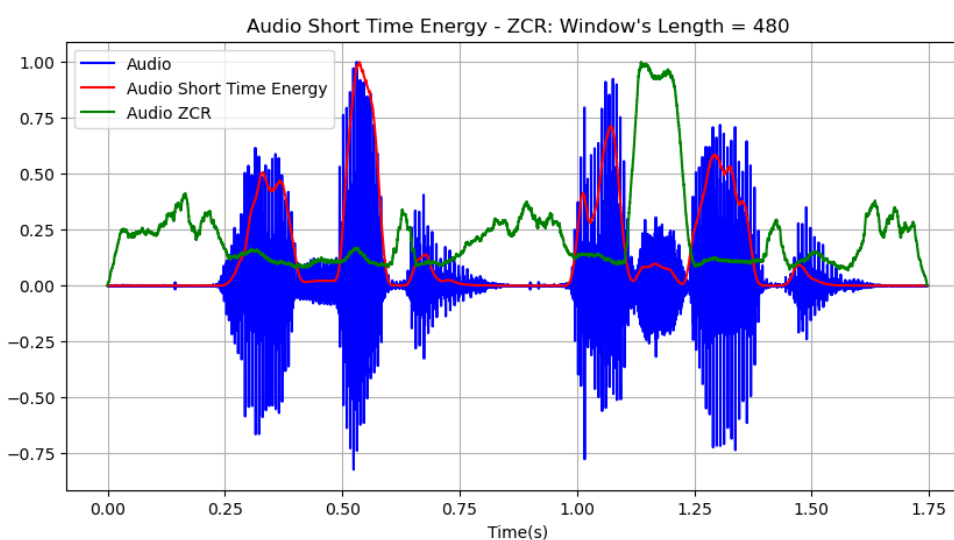
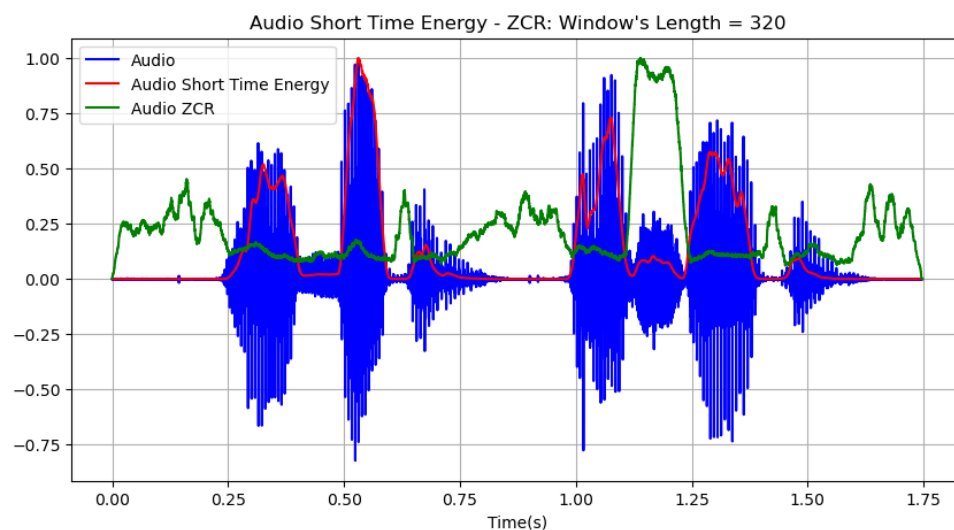
Question 2.2

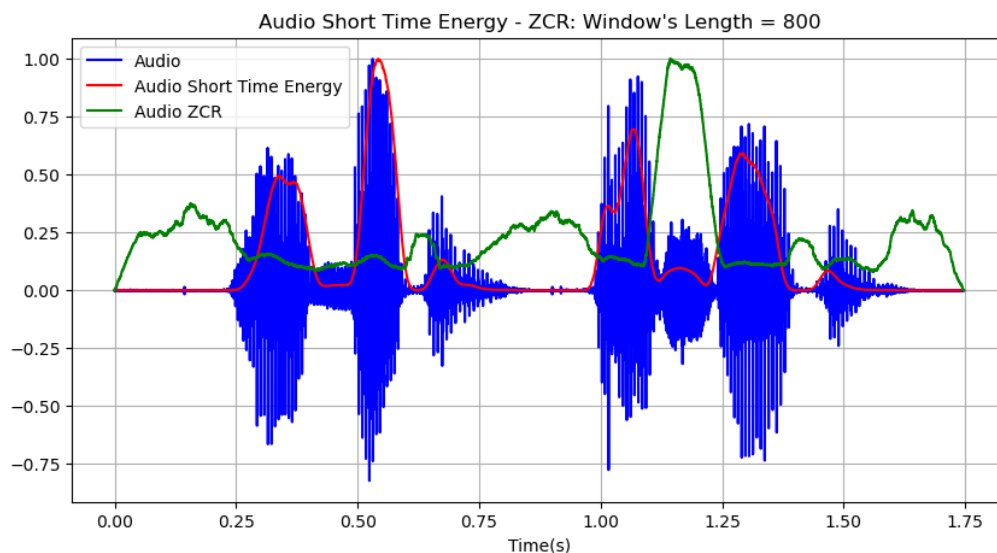
We will calculate the **short time energy (E_n)** and **sign exchange rate (ZCR)** of the signal for window values **of 20-50ms** per 10ms, so **length $L = 0.02 * f_s = 320$ to $L = 0.05 * f_s = 800$** . Short-time energy and ZCR are given by the formulas:

$$E_n = \sum_{m=-\infty}^{m=\infty} |x[m]|^2 \square w[n - m]$$

$$Z_n = \sum_{m=-\infty}^{\infty} |\text{sgn}(x[m]) - \text{sgn}(x[m - 1])| \square w[n - m]$$

After making the windows **hamming** (for E_n) and **rectangular** (for Z_n) we create these sizes using **convolution** and specifically the **convolve** function of scipy.signal library. The shifted signals, where needed, were calculated using **the roll(signal,n)** of numpy which **shifts cyclically** by **n**. We list the sizes for the different windows in a common diagram below:





We observe a ***gradual smoothing of the*** En and ZCR graphs with the increase in window size as changes in the signal are smoothed out faster. This provides us with a slightly ***less detailed picture*** of energy but ***cuts out noise***, thus providing a more realistic picture of the energy and signal change rate

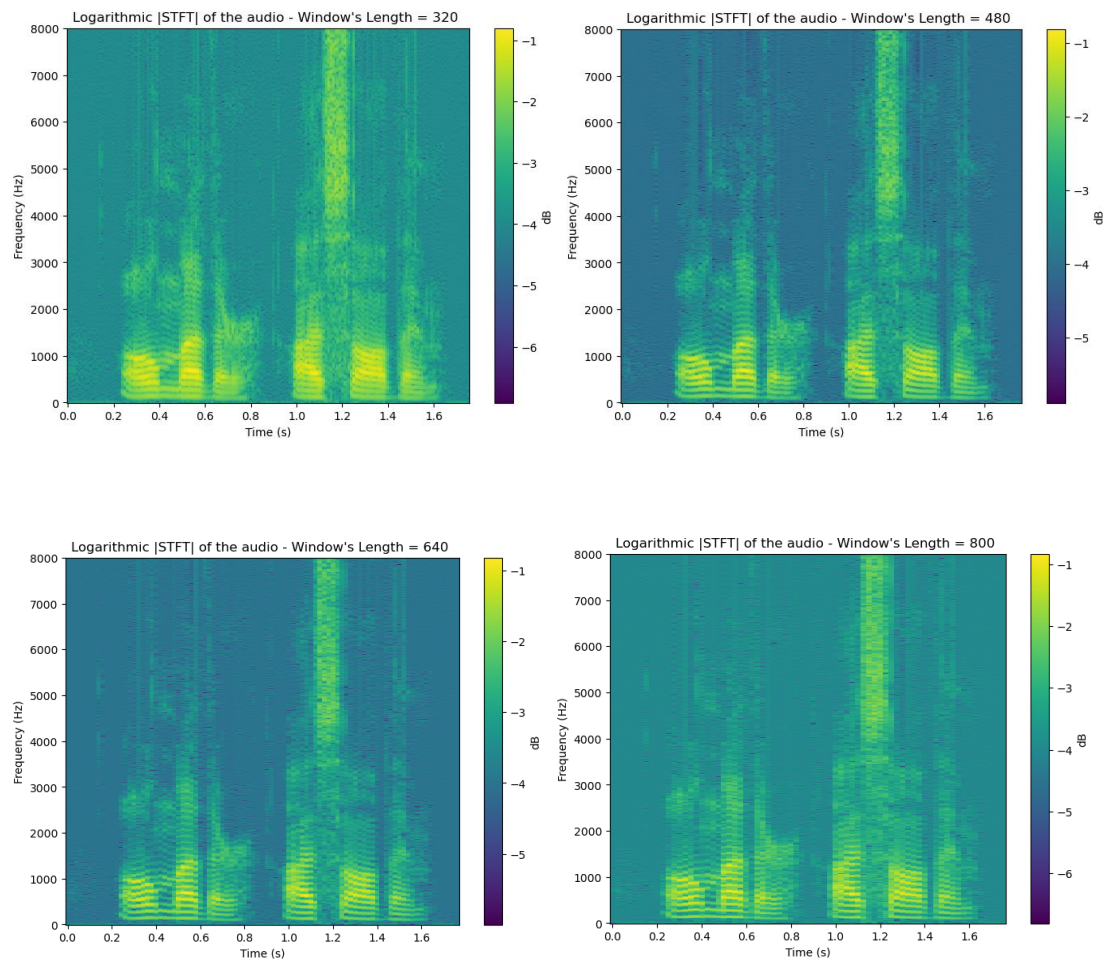
These measurements would be useful for further processing of the voice signal. More specifically, we could separate the time intervals in which there is ***silence*** from those that contain ***sound***, as the former correspond to ***points of very low to zero energy*** and the latter to ***points of low zero crossing rate***.

In addition, these diagrams provide us with a good insight into ***identifying voiceless*** ("Oh Mother, I saw them") or ***voiceless*** sounds. The ***voiceless*** sounds seem to be in the areas where the ***energy is at peak*** and the ***zero crossing rate is low***, while the ***voiceless*** ones in ***low energy areas and at the peaks of the zero crossing rate***.

Question 2.3

For the various window length values mentioned above, we calculate the ***Short Time Fourier Transform*** which gives us an insight into the ***time-frequency relationship*** through the **`sp.signal.stft()`** function of scipy and obtain the graph of the ***logarithm*** of this measure (adding a constant of

0.0001 for better visualization). Here are the graphs of the STFT measure that we made using *pcolormesh()* Matplotlib's:



STFT on a logarithmic scale for the various window length values.

Conclusions:

Energy: Areas of *intense color* are *high-energy intervals* and even coincide with the corresponding intervals of the Short Time Energy chart.

Time: We observe that in the first periods of the signal low frequencies predominate *while in the middle and towards the end participate the higher ones*.

Frequency: We seem to have more energy at *low frequencies* (more intense color)

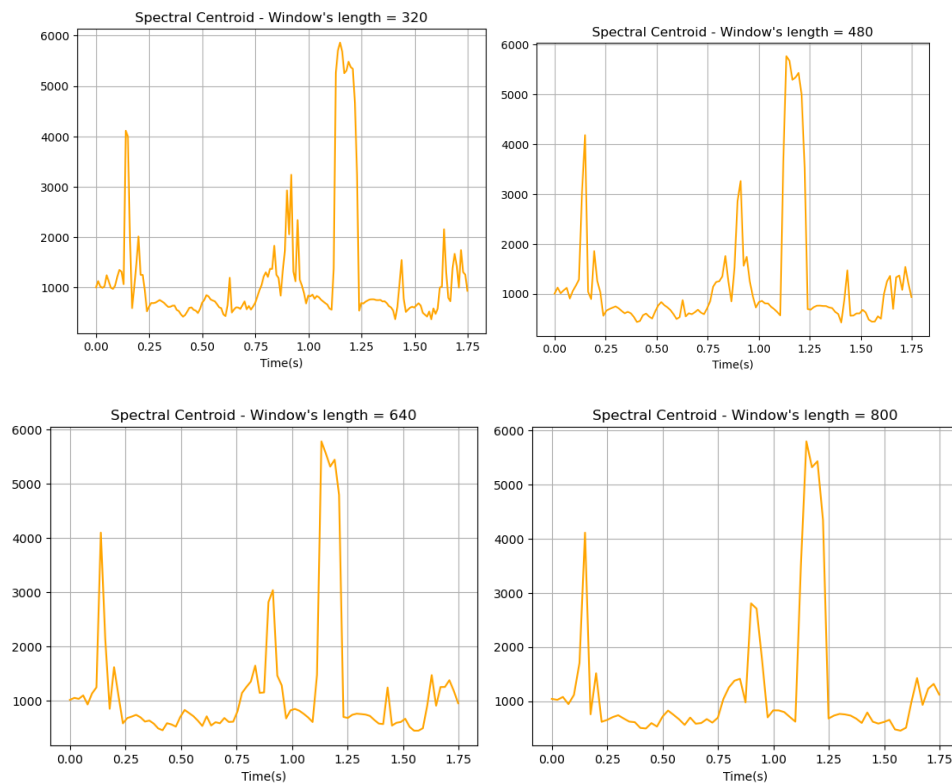
Windows: During the gradual increase of window lengths we observe a slight *distortion-fading* of the color which is due to the *removal of noise*

and the small energy it adds to our basic voice signal) for the calculation of which we constructed functions that "build" the following formulas of the following formulas

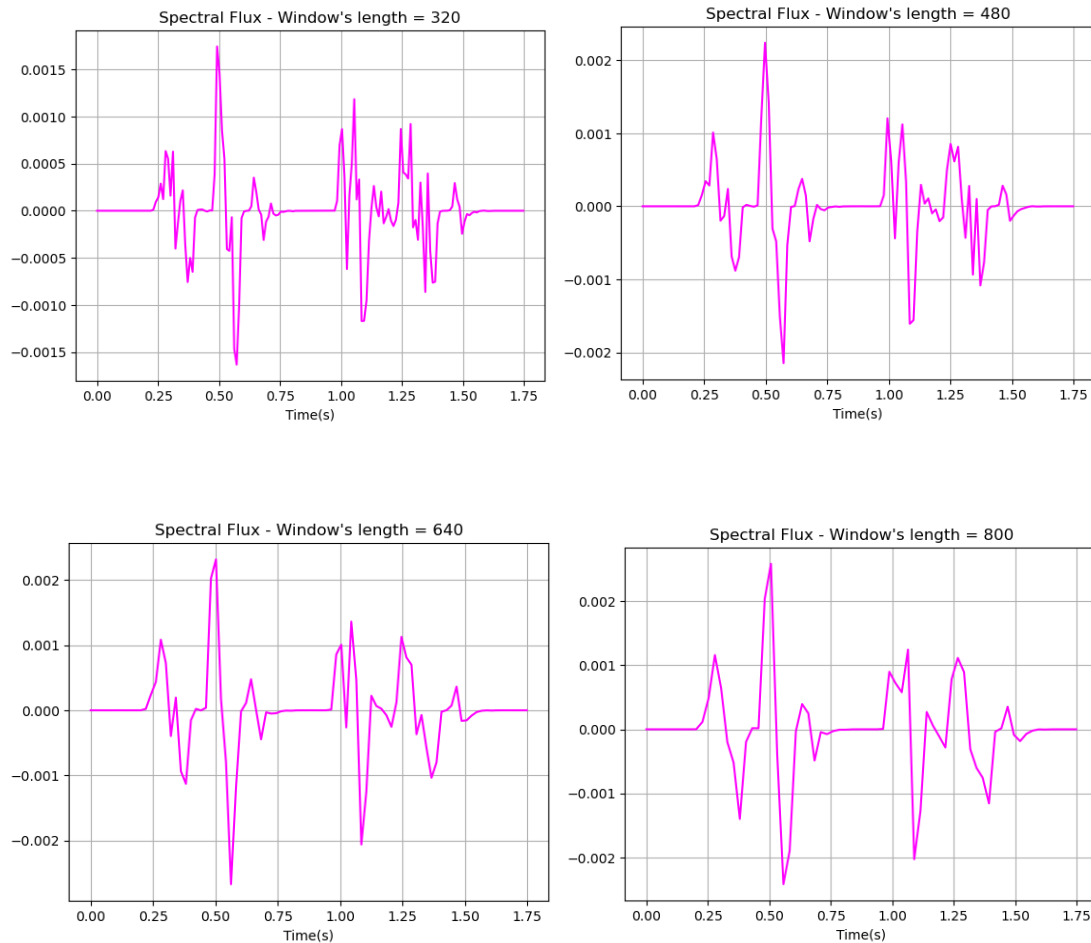
$$SC = \frac{\sum_{k=0}^{\frac{N}{2}} k |X[k, m]|^2}{\sum_{k=0}^{\frac{N}{2}} |X[k, m]|^2} \cdot \frac{fs}{N}$$

$$SF = \left| \left| \frac{|X[k, m+1]|^2}{\sum_{k=0}^{\frac{N}{2}} |X[k, m+1]|^2} - \frac{|X[k, m]|^2}{\sum_{k=0}^{\frac{N}{2}} |X[k, m]|^2} \right| \right|$$

We list the **spectral center** and **spectral flux** graphs for the different window length values:



Spectral center for the various window length values.



Spectral flux for the various window length values.

The ***spectral center*** represents the "center of gravity" of the signal strength spectrum and actually gives us an idea of where the ***main energy point of the signal is*** located.

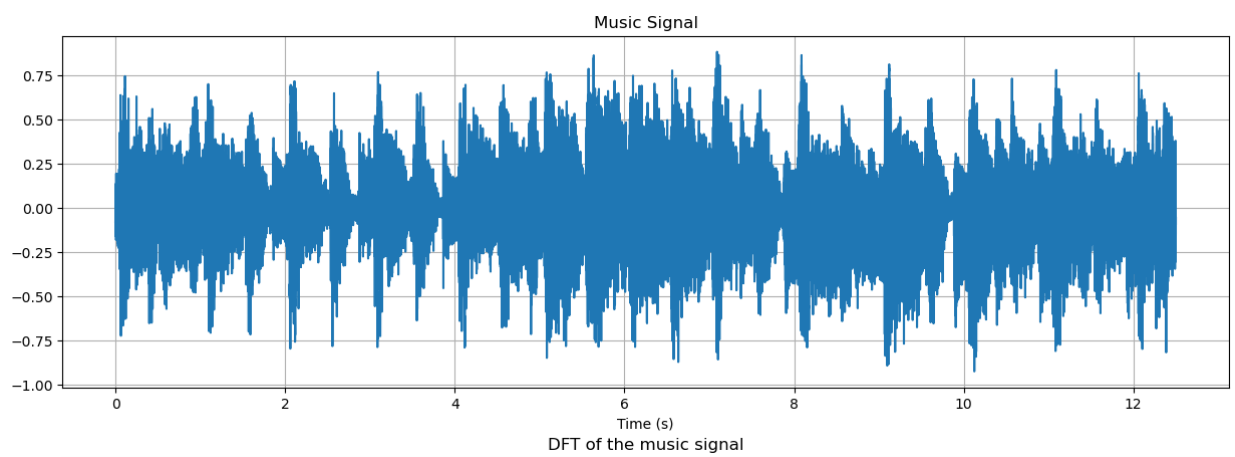
Spectral ***flow*** measures the change in the sound spectrum over time, i.e. it can be used to detect ***changes in sound composition***. For this reason we have peaks (voiceless-voiceless sounds) and zero value between (pauses between words).

In these two physical quantities we observe ***a gradual smoothing*** of the signal with an increase in the length of the window as ***noise decreases*** and ***more points are covered over time***. Also, there is an ***increase in spectral flux***

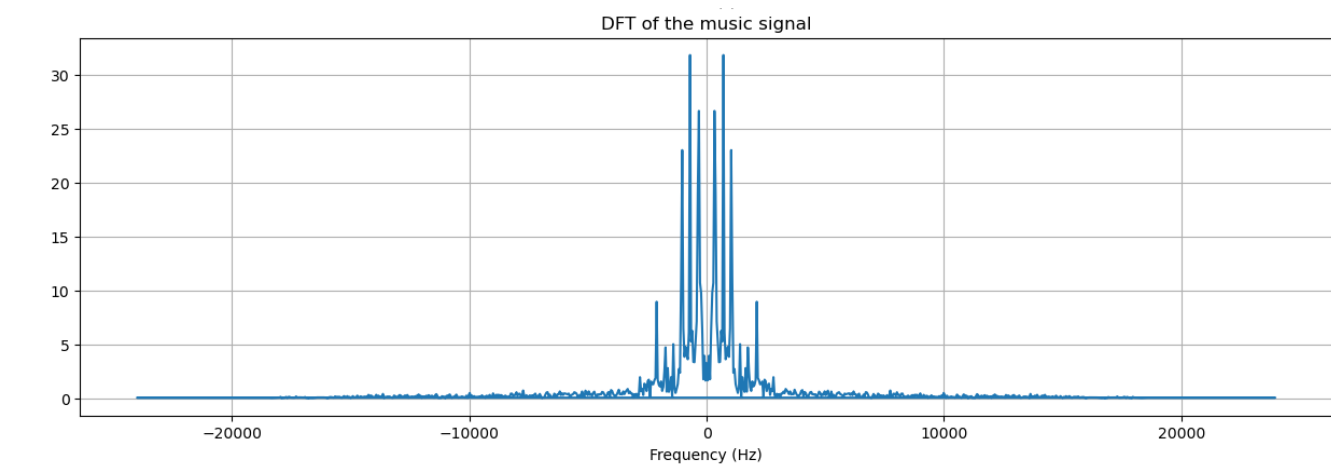
Question 2.4

We repeat the above questions for the music signal "*music.wav*" sampling frequency $fs=48kHz$.

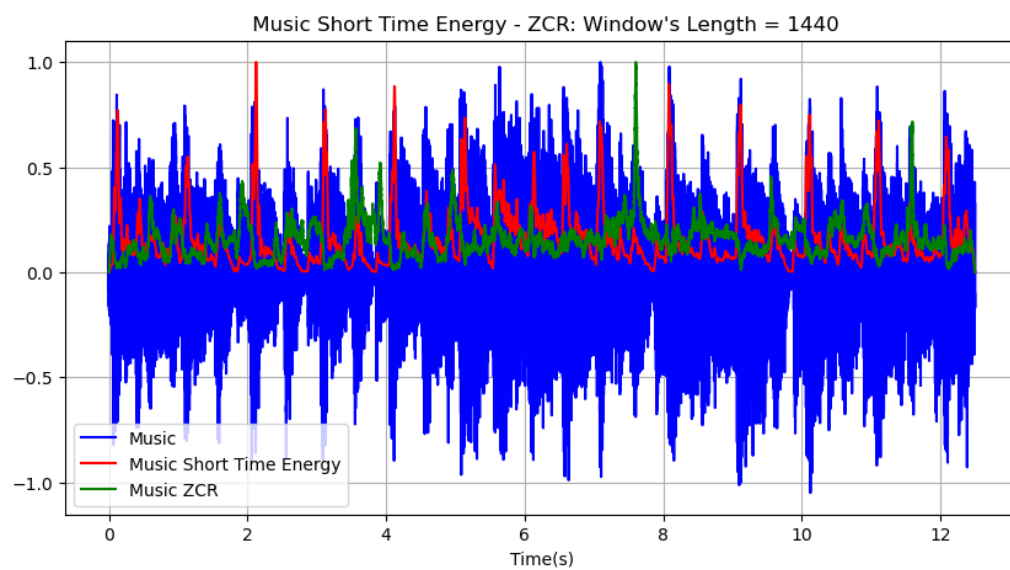
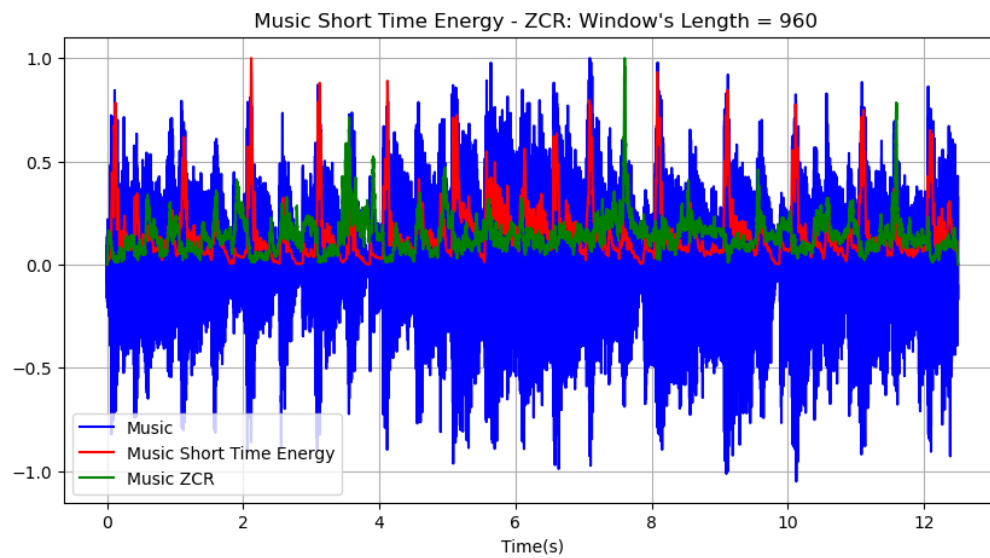
Music signal in time:

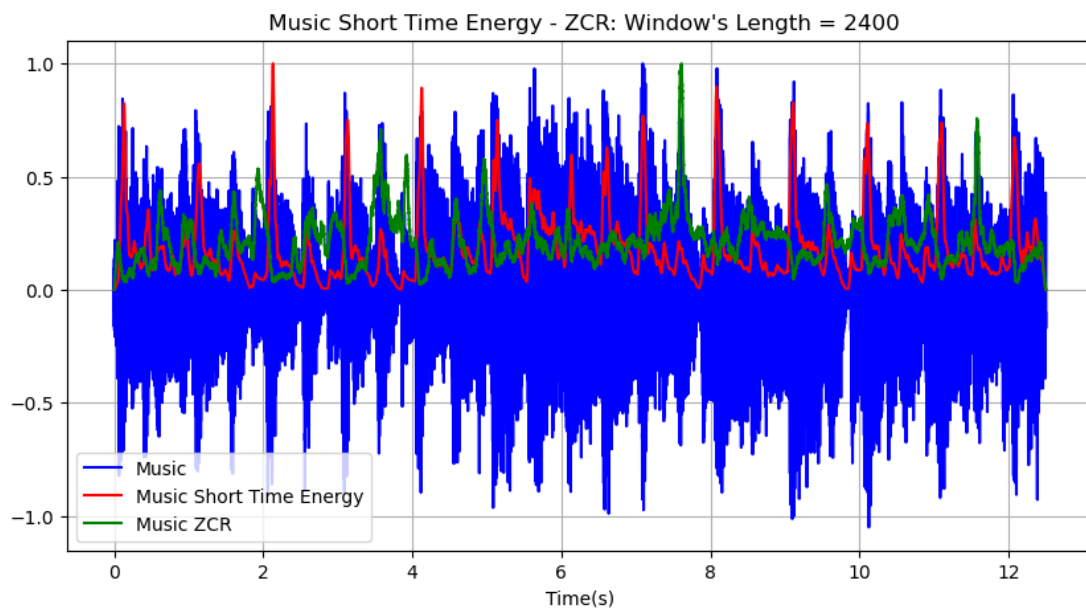
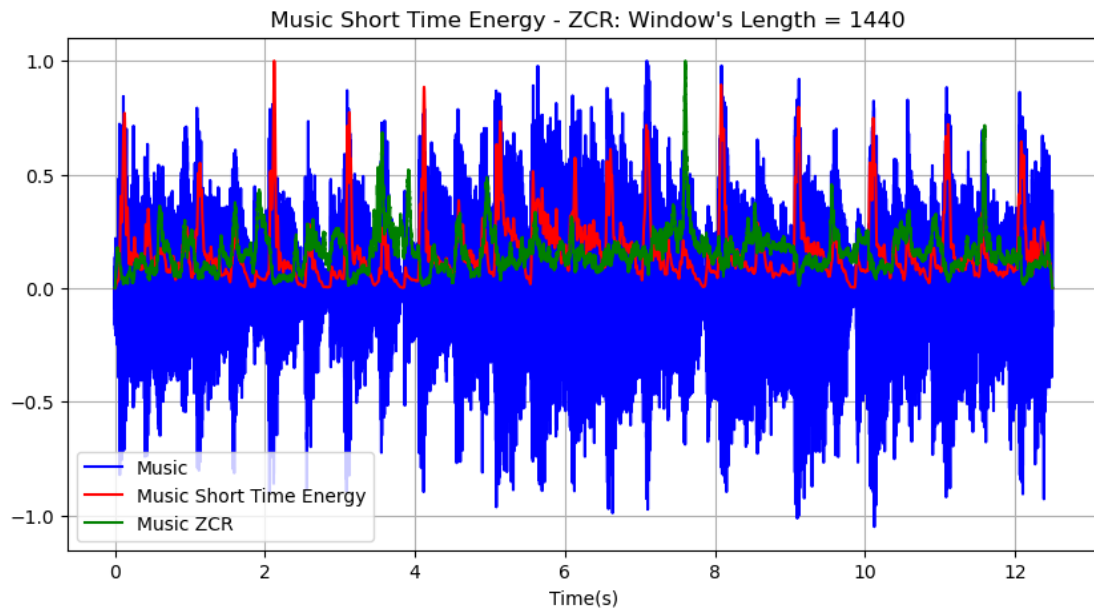


DFT of the mark:



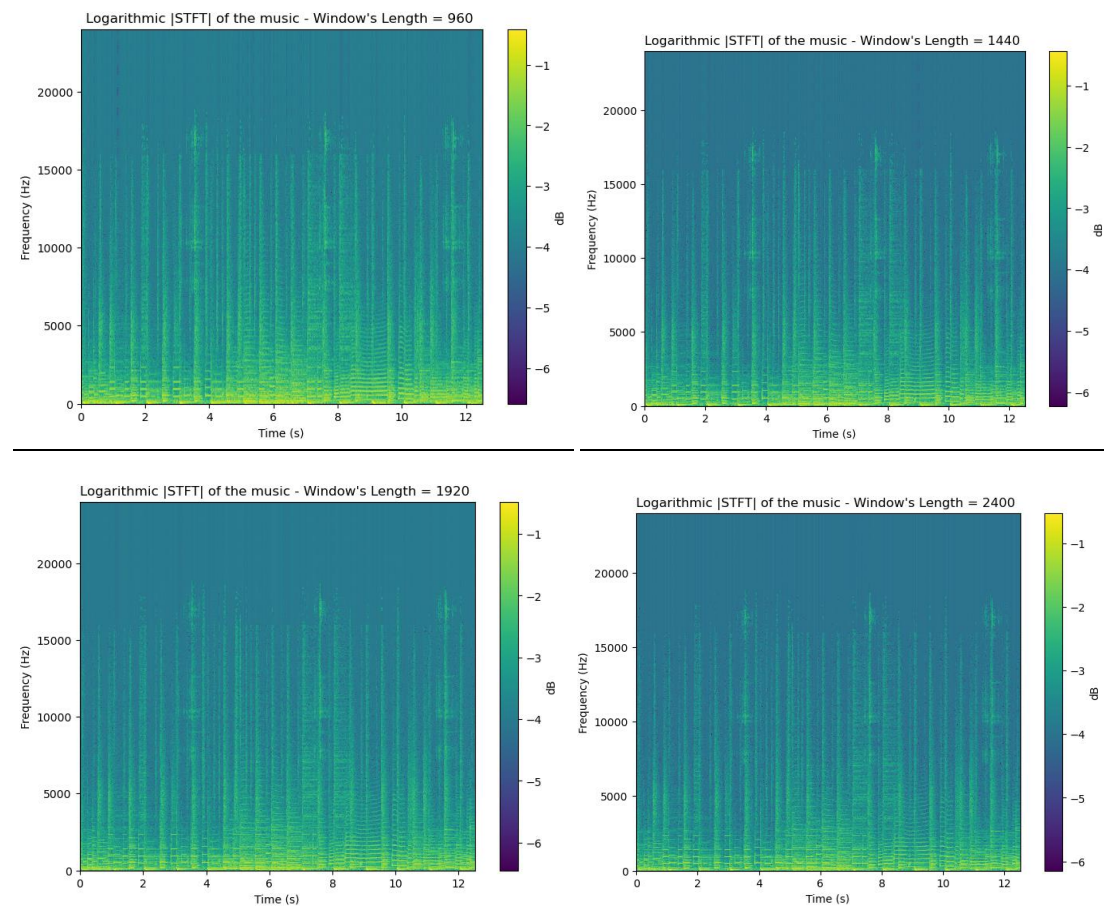
Short time energy and ZCR of music signal for various windows:





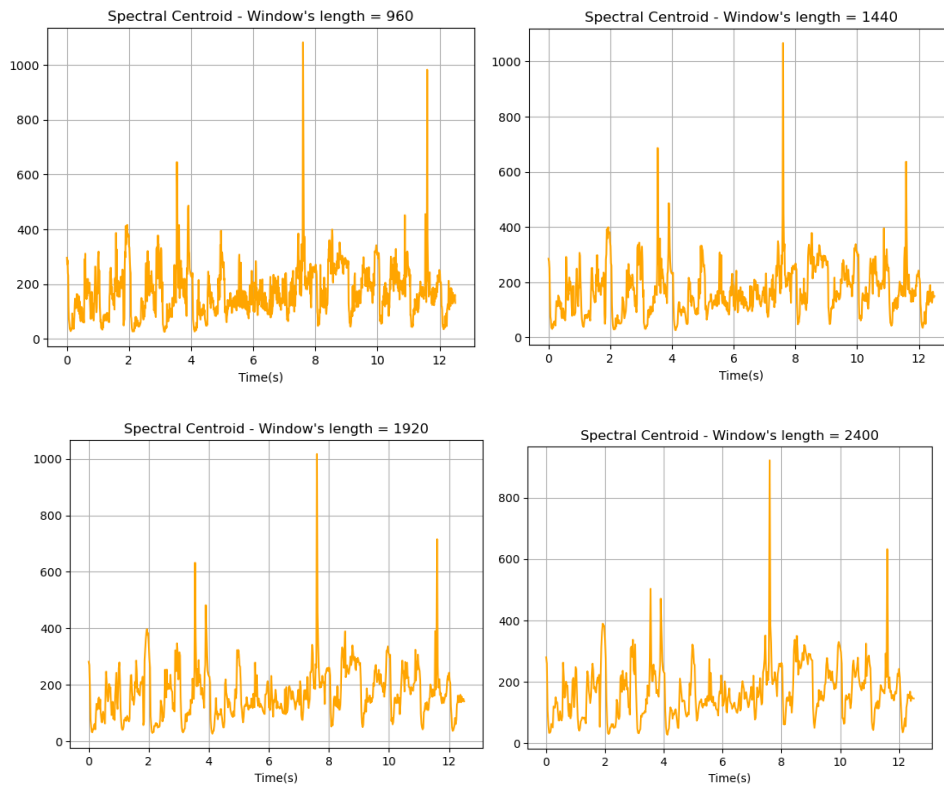
We notice smoothing with increasing length of windows. In addition, the signal seems to contain several sounds with depth as its energy makes many peaks (as before in energy peaks we have low ZCR while this makes peaks in shallower sounds).

STFT on a logarithmic scale for the music signal:

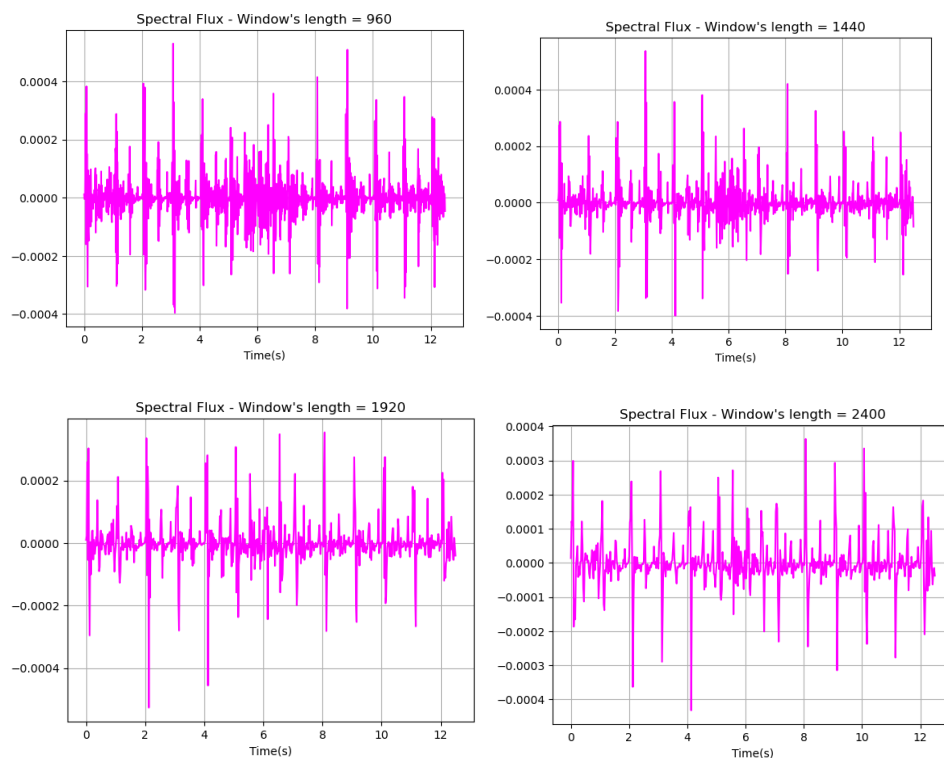


The signal appears to have an ***even distribution of energy in the time domain*** (x-axis) while **more energy appears to be in the lower frequencies** (low on the y-axis).

Spectral center of the music signal for the different values of the window:



Spectral Music Signal Stream for different window lengths:



The smoothing is evident by increasing window lengths, providing us with a clearer view of both SC-SF sizes.

Part 3 - Digital Processing and Spectral Analysis of Cetacean Biosignals

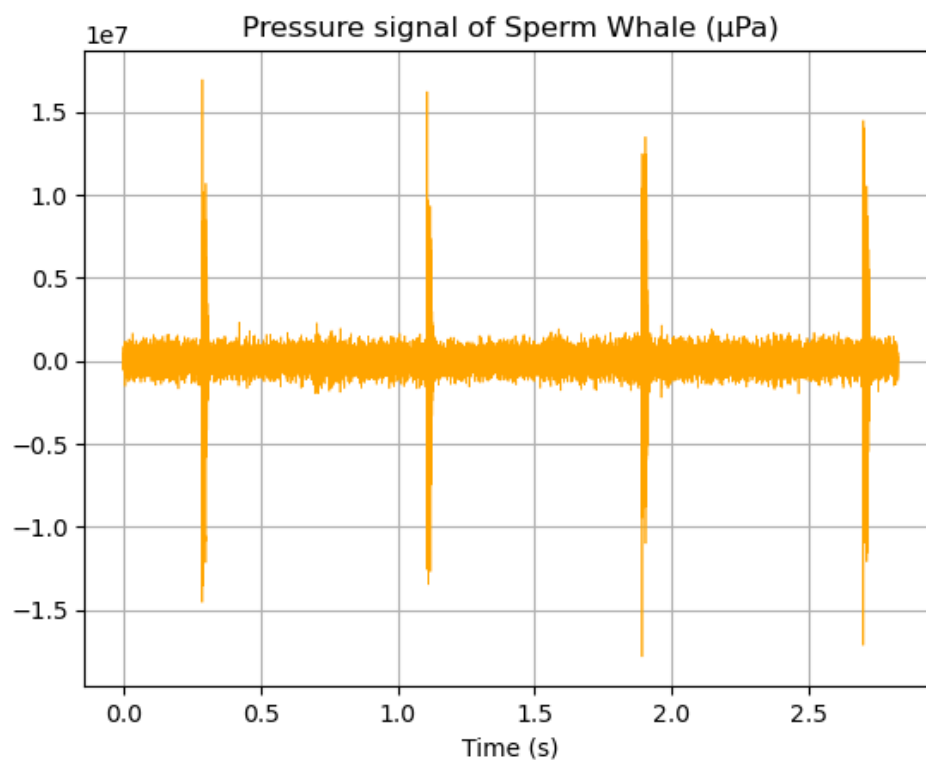
●The purpose of the exercise is the **digital processing of two whale biosignals** collected by hydrophones. These are signals from whales of two species: Sperm whale and Humpback whale. Finally, it is studied how anthropogenic sound can affect these whales.

Question 3.1

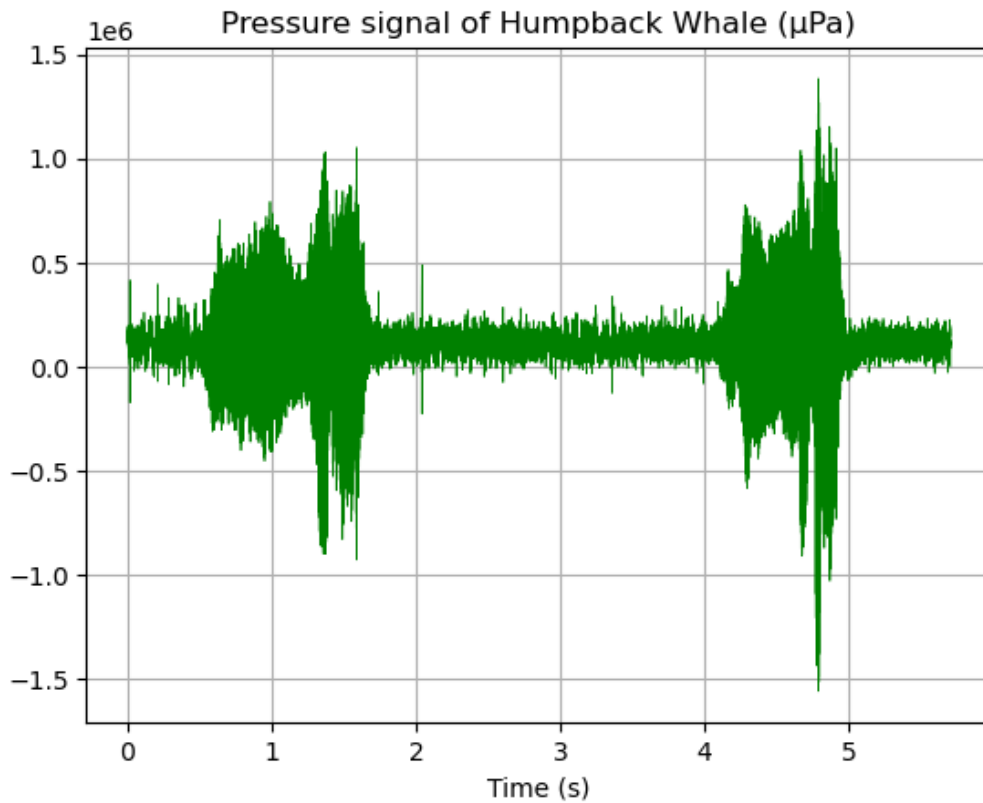
First we convert biosignals from voltage signals to pressure signals according to the following formula:

$$pressure = \frac{3.5 \times y \times GF}{10^{sensitivity/20}}$$

Where y is our signal which we load with the help of `load()`, GF the amplification factor and $sensitivity$ the sensitivity of the hydrophone. Given **values** $GF=0.16$ and **sensitivity** = **-155 dB** relative to 1V/μPa we obtain the following pressure-time diagrams:



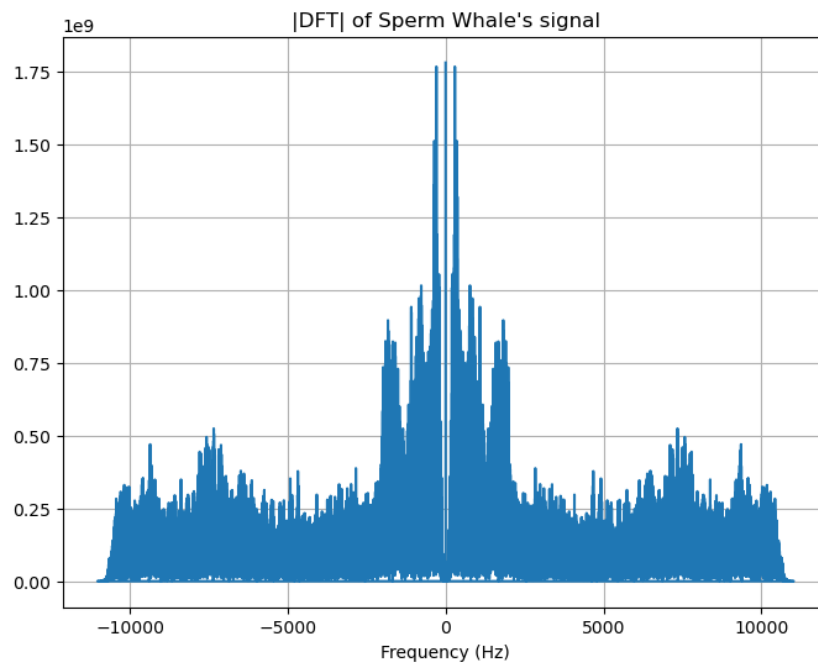
Sperm whale pressure signal.

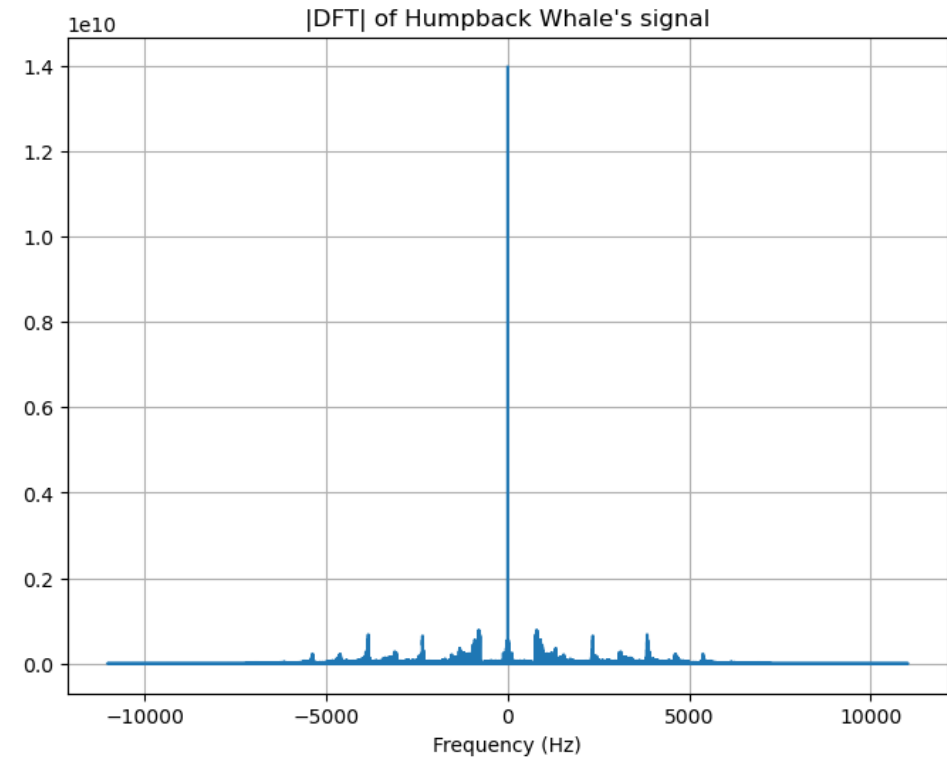


Humpback whale pressure signal.

We also found that the two signals have the same **sampling rate** $f_s=22050\text{ Hz}$.

In addition, we present the **DFT's** of these brands:





Question 3.2

Then we will calculate the rms pressure value as well as the sound pressure level (SPL) based on the following formulas:

$$Prms = \sqrt{\frac{1}{N} \sum_{i=1}^N pressure^2}$$

$$SPL = 20 \log\left(\frac{Prms}{Pref}\right)$$

Where N is the length of the signal and Pref = 1μPa.

The message printed on the screen is as follows:

```
RMS of Sperm Whale signal is RMS1 = 749840.3376660865 .
RMS of Humpback Whale signal is RMS1 = 195774.37361509659 .
SPL of Sperm Whale signal is SPL1 = 117.49937599177156 dB relative to 1 μPa .
SPL of Humpback Whale signal is SPL2 = 105.83511686211784 dB relative to 1 μPa .
```


Question 3.3

We choose the *humpback whale* signal and confirm *Parseval's theorem*, which states that energy in frequency space is equal to this calculation in the time domain:

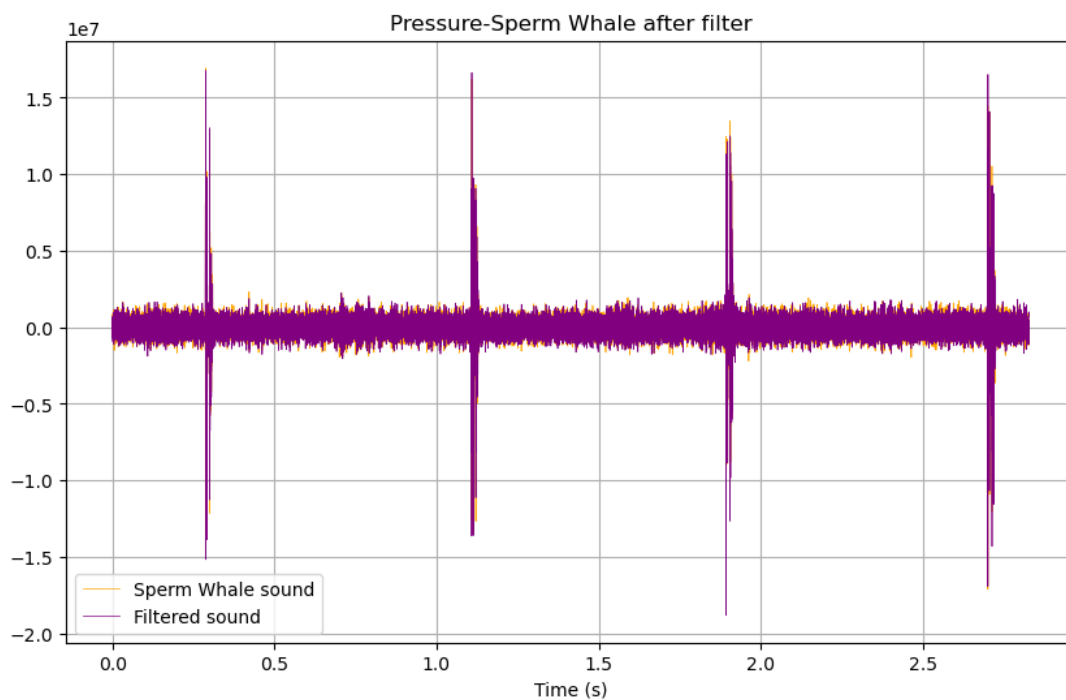
$$E_n = \sum_{n=0}^{N-1} |p[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |P[k]|^2$$

The following message is printed which means that the theorem is verified (The small difference is due to any approximations or errors):

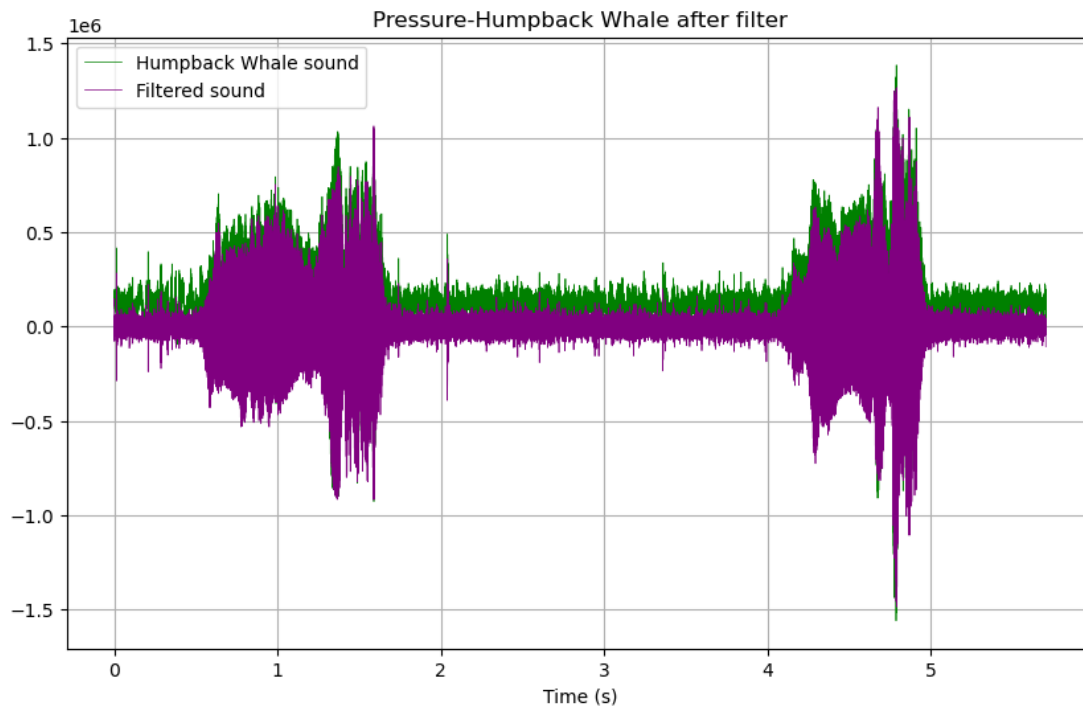
```
Energy in time domain is 35043394 .  
Energy in frequency domain is 35043447 .
```

Question 3.4

We will **isolate** from pressure biosignals the **anthropogenic** noise of low frequency, mainly ships, by applying a **high pass filter** $W_n=200/(fs/2)Hz$ and **class** $N=3$ which will isolate frequencies below 200Hz ($fs/2$ frequency *Nyquist*). We do this using *scipy.signal.butter()* and *scipy.signal.lfilter()*. The results are shown below:



Filtered Sperm whale signal.



Filtered Humpback whale signal.

It is obvious that our filter has ***limited the low frequency*** of our biosignals while keeping frequencies above 200Hz ***almost unchanged***.

Question 3.5

Knowing that ***humpback whales*** hear frequencies between ***300Hz-3000Hz*** while ***sperm whales hear the full range*** and furthermore that their behavior is affected for SPLs ***greater than 100dB***, we will examine whether the man-made sound "Pile driving" is capable of negatively affecting whales. More specifically, for sperm whale we calculate the SPL of the Pile driving signal, after that too is first converted into a pressure signal, and we calculate its SPL. Similarly, for Humpback, we filter the anthropogenic signal with a ***bandwidth filter*** $Wn=[300/(fs/2), 3000/(fs/2)]$ where $fs/2=11025\text{Hz}$ is the ***frequency Nyquist***. The result printed on the screen is as follows:

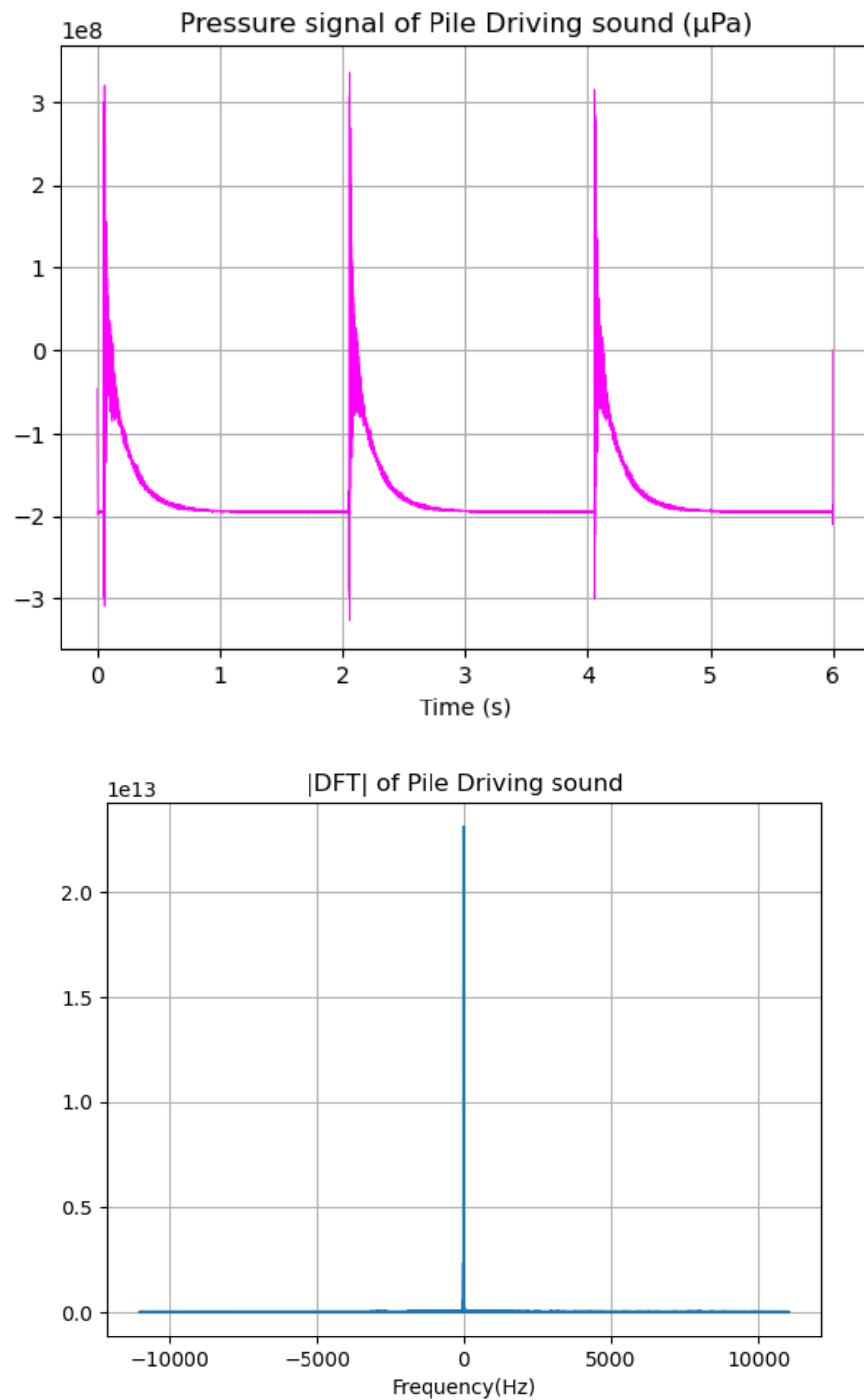
Sampling rate of Pile Driving signal is $fs3 = 22050$

Sperm Whale: Sound Pressure Level of Pile Driving sound is $SPL3_sperm = 165.15296950722166 \text{ dB}$.

Humpback Whale(Filterd Pile Driving): Sound Pressure Level of Pile Driving sound is $SPL3_hump = 141.51802420615306 \text{ dB}$.

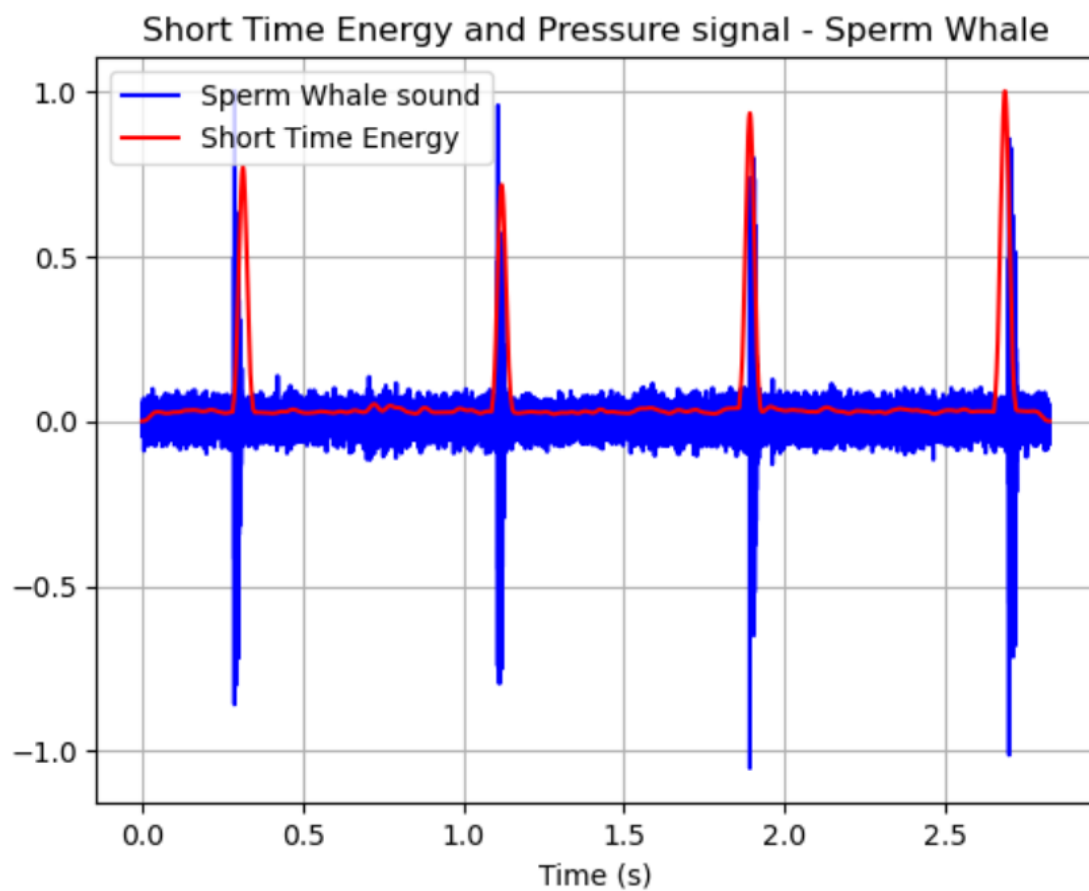
Therefore, the anthropogenic Pile driving signal ***is considered harmful for both whale species*** since the SPL sound pressure level of filtered and unfiltered signal ***exceeds 100dB***.

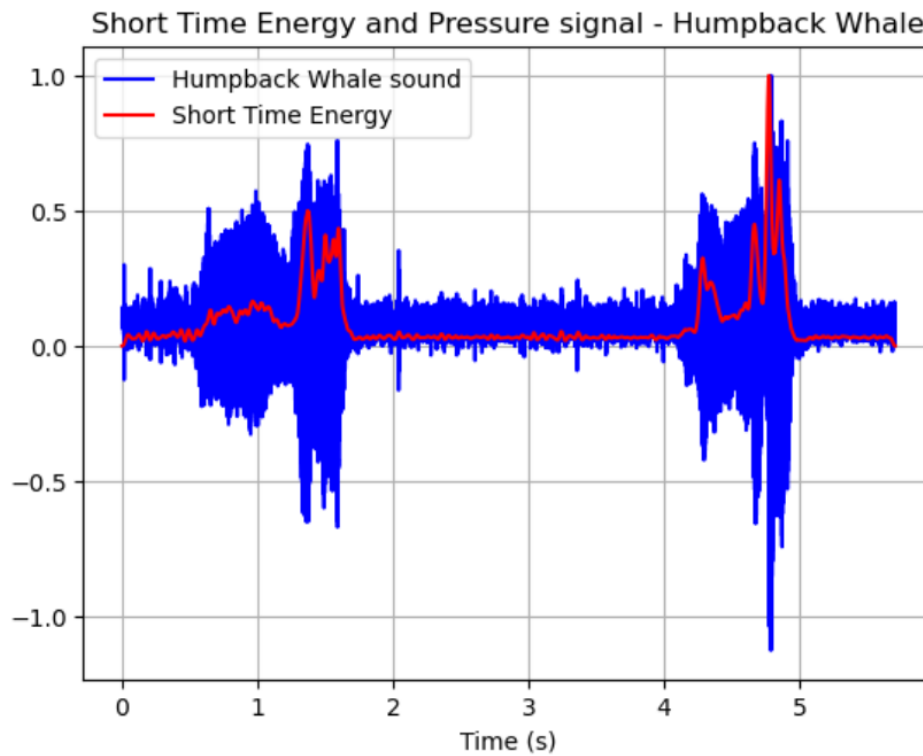
Finally, we present the **pressure graph** of the unfiltered Pile driving signal as well as its **DFT** for a complete picture of the signal:



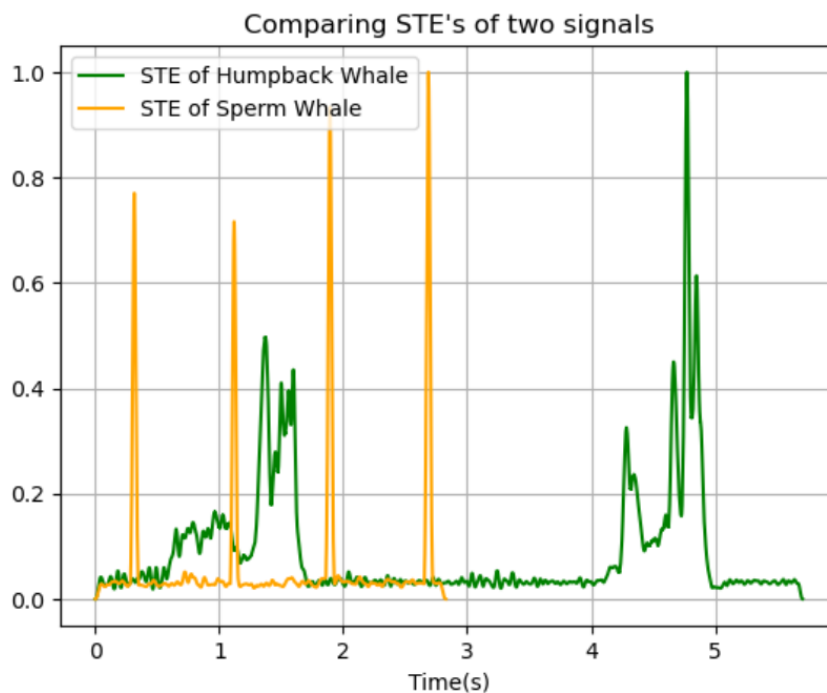
Question 3.6

As in previous exercises of the current report, we will calculate the **short-time energy** of the two biosignals for window hamming length $L=0.05s$ $O22050Hz = 1102$ as convolution of the square of each biosignal with the window using **convolve()**. After normalizing for best visual results, we present the pressure and energy graphs for each whale and the two short-time actions between them:





Comparing STE's of two signals



In the first two graphs, at the points of voiceless sound the energy is almost zero, while at the points where the sound is phonic we have peaks. Also, we observe that the sperm whale produces a biosignal ***of much higher energy*** than the humpback whale since its peaks are much

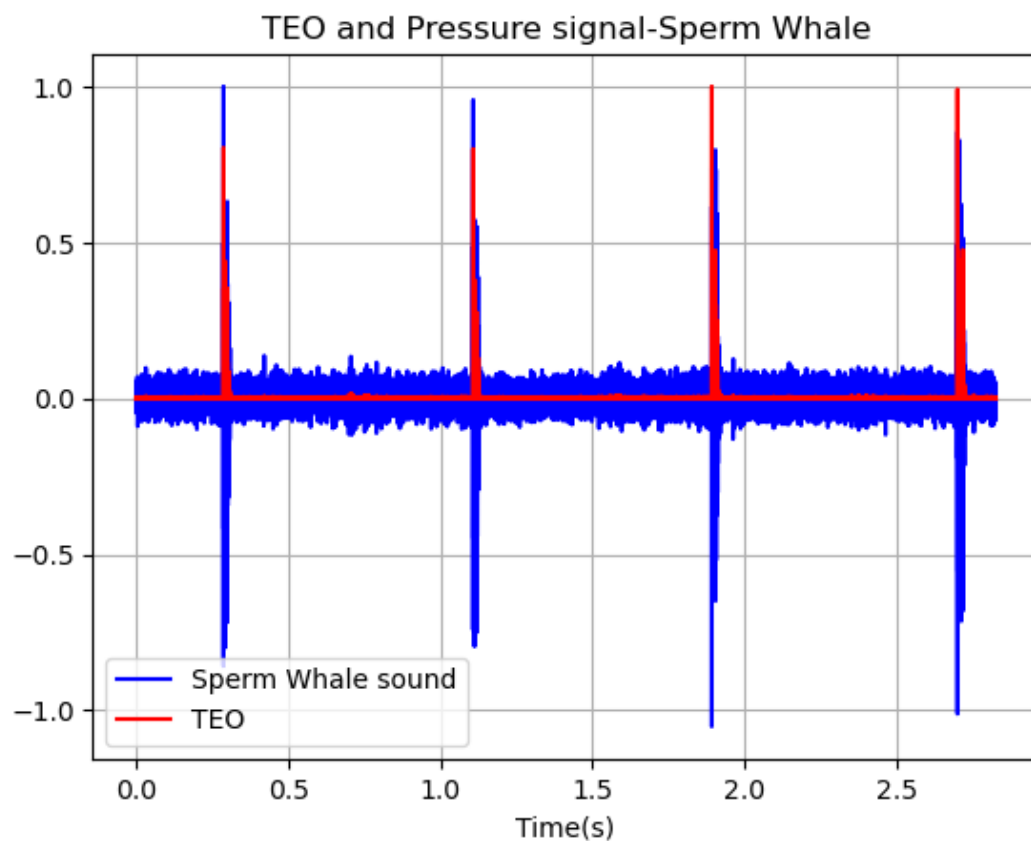
higher than the peaks of the humpback (we confirmed this from the formula). $En = \frac{1}{N} \sum_{k=0}^{N-1} |P[k]|^2$

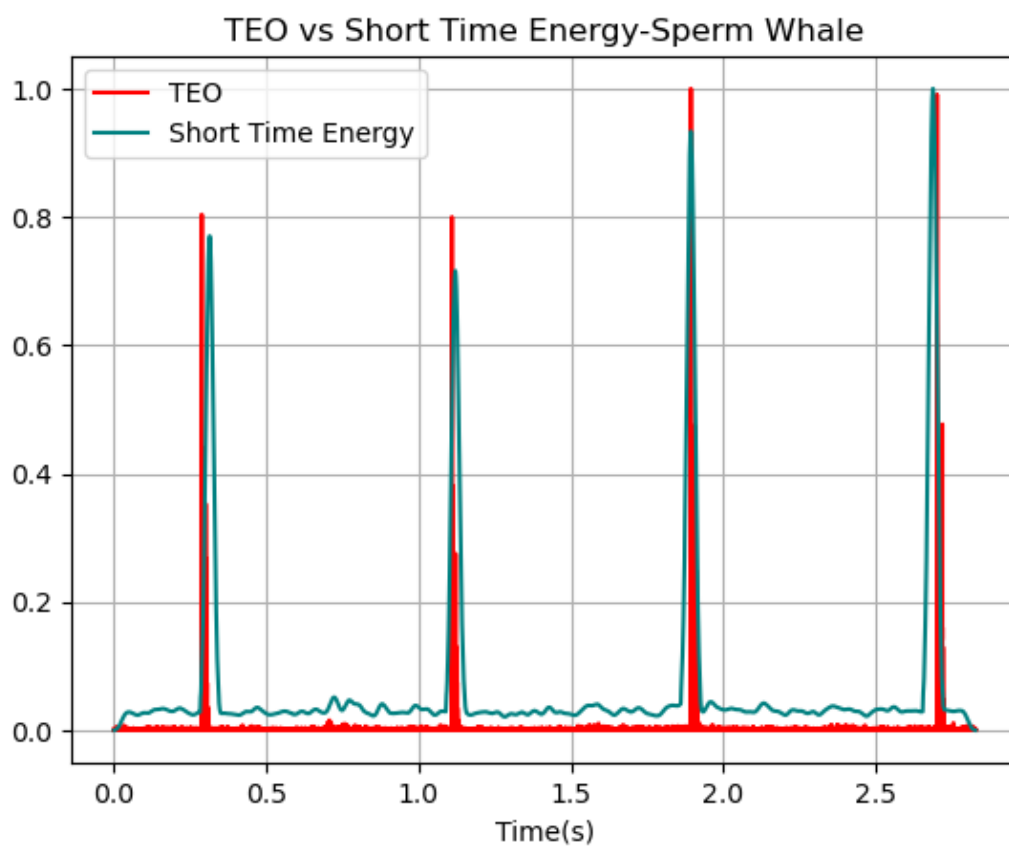
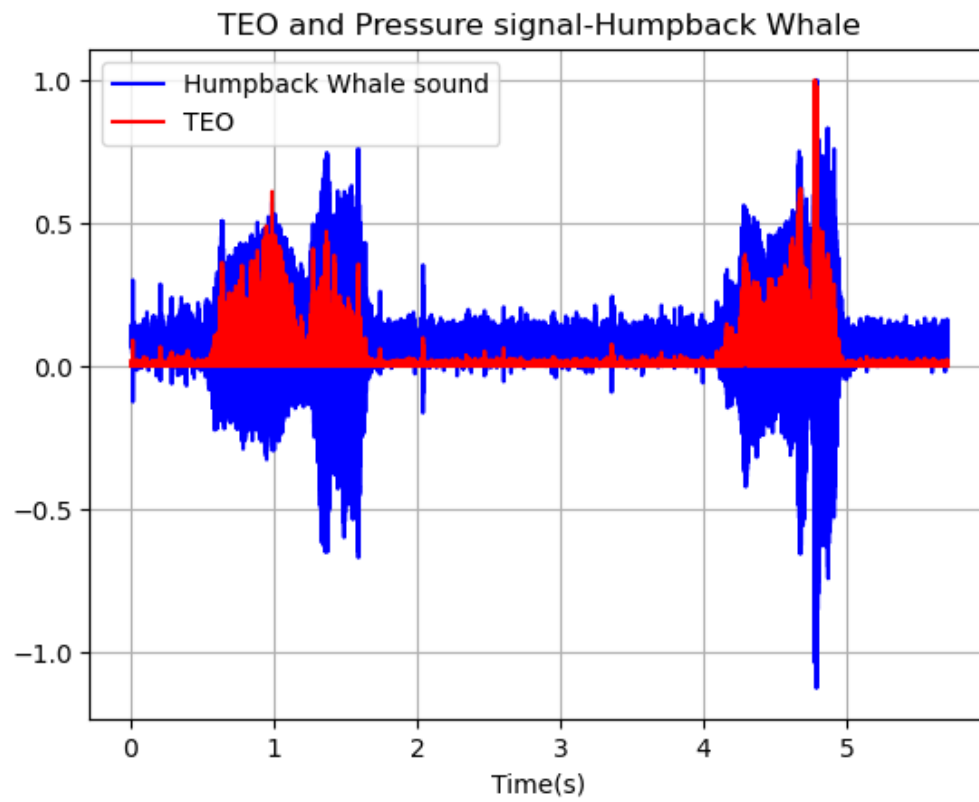
Question 3.7

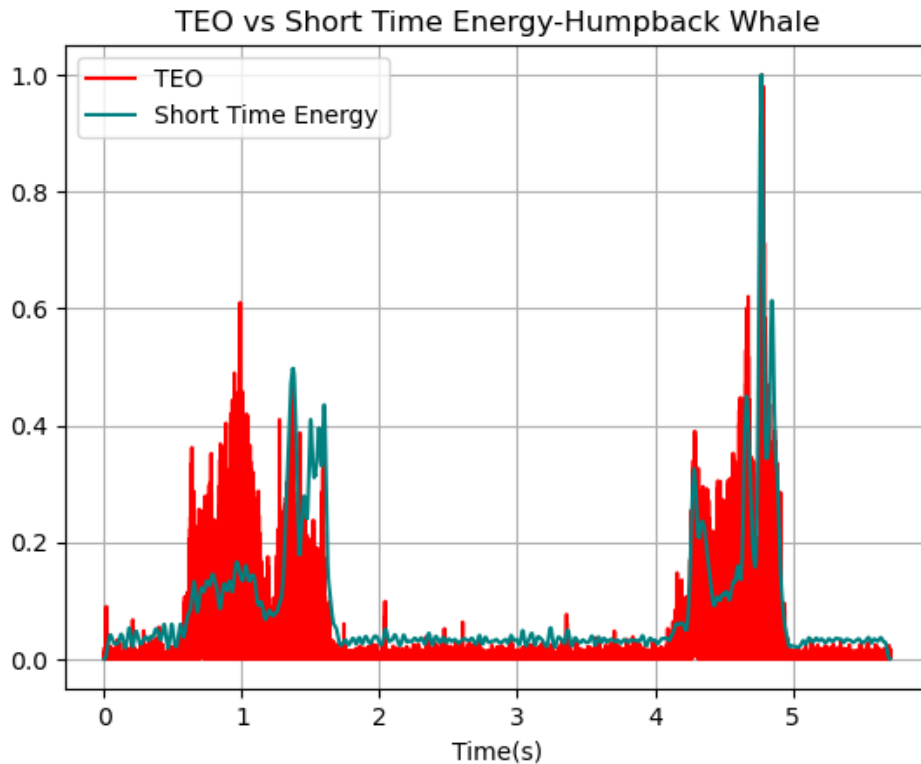
We will implement the nonlinear **Teager Kaiser Energy Operator** (TEO) operator for the two biosignals which results from the following formula:

$$\Psi[p[n]] = p[n]^2 - p[n-1] \cdot p[n+1]$$

After constructing it with the help of the circular shift **np.roll()** where needed (), we draw its graph and compare it with the short-time energy: $p[n-1] \rightarrow 1$ θέση δεξιά, $p[n+1] \rightarrow 1$ θέση αριστερά



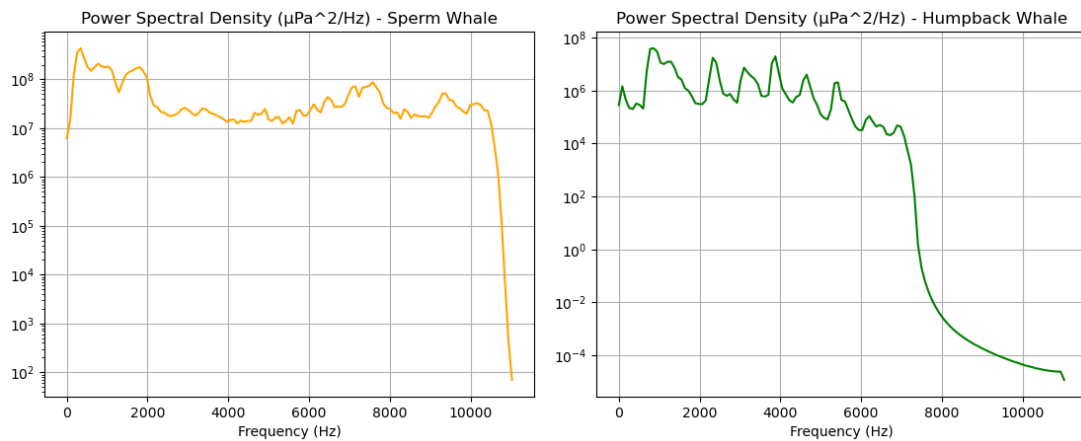




We notice that TEO is slightly higher than short-time energy in some places. According to information we searched online, TEO is generally an ***indicator of energy exchange detection and is useful in signals with enough noise*** like ours. So we observe that it follows the form of STE but in places with many variations of it, It becomes even bigger.

Question 3.8

We will plot the ***Welch periodogram*** on a logarithmic scale using ***.welch()***. This is an estimate of the spectral density of power and will give us an illustration related to the **energy in the spectrum**. We list the periodogram for the two biosignals below:



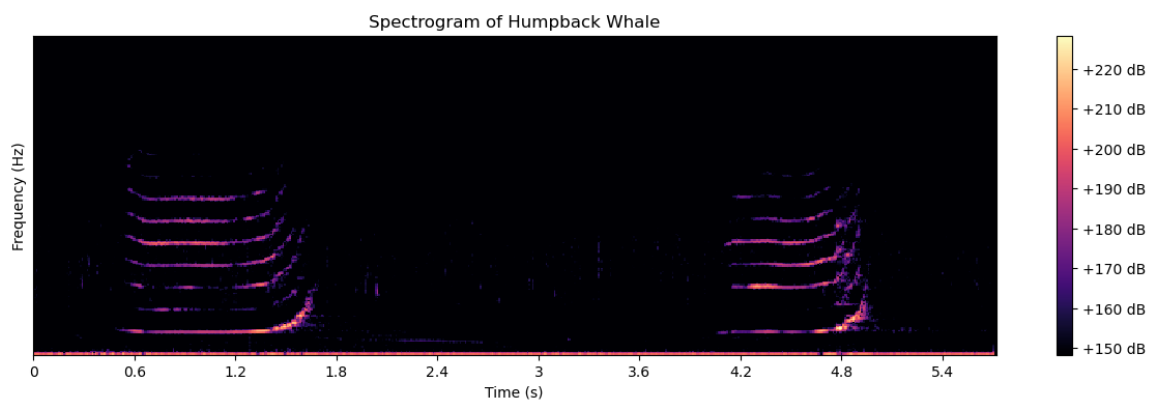
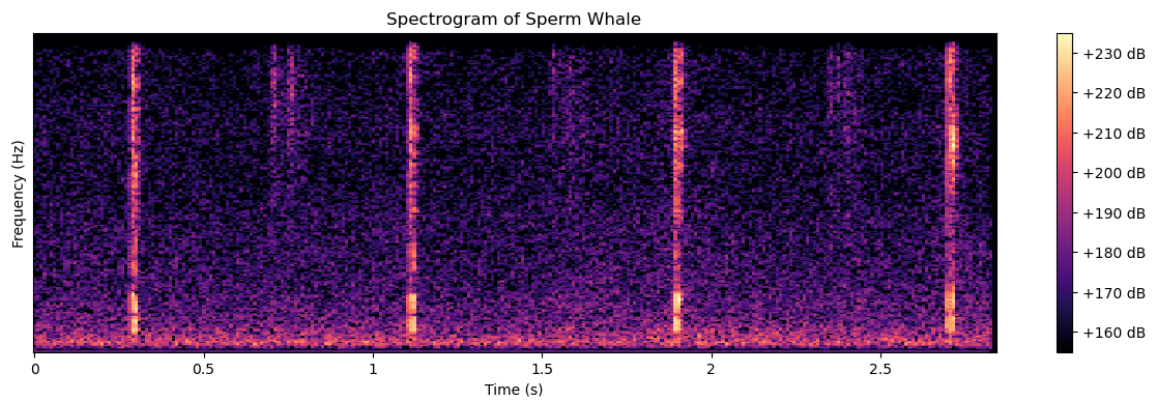
We observe that the spectral power density in both signals ***is attenuated at high frequencies*** while it is ***strong at smaller frequencies***.

In addition, in the sperm whale we have peaks of spectral density – and therefore energy – for a wider frequency range than in the humpback. (0 to 9500Hz vs 0 to 6500Hz)

Finally, it gets higher values in the biosignal of the Sperm whale (up to over 10^8) than in the Humpback (up to a little less than 10^8). This is not something we didn't expect since ***the sperm whale showed above that it has more energy*** and ***the above diagrams are directly related to the energy distribution of the two signals in the spectrum***.

Question 3.9

We will design the spectrogram of the squares of the two biosignals with the help of ***librosa.display.specshow()*** after calculating the ***STFT*** of each of them (***sp.signal.stft()***) and converting the result to a logarithmic scale using ***librosa.amplitude_to_db()***.



As we saw in the power spectral density diagrams of the *previous question*, the humpback whale has a much lower energy, which is indicated by the *less intense colors in the spectrogram* (see colorbar).

Time field: We observe *high energy at the same intervals we already saw in the short-time energy graphs* and an absence of energy at the corresponding intervals, which we expected to see.

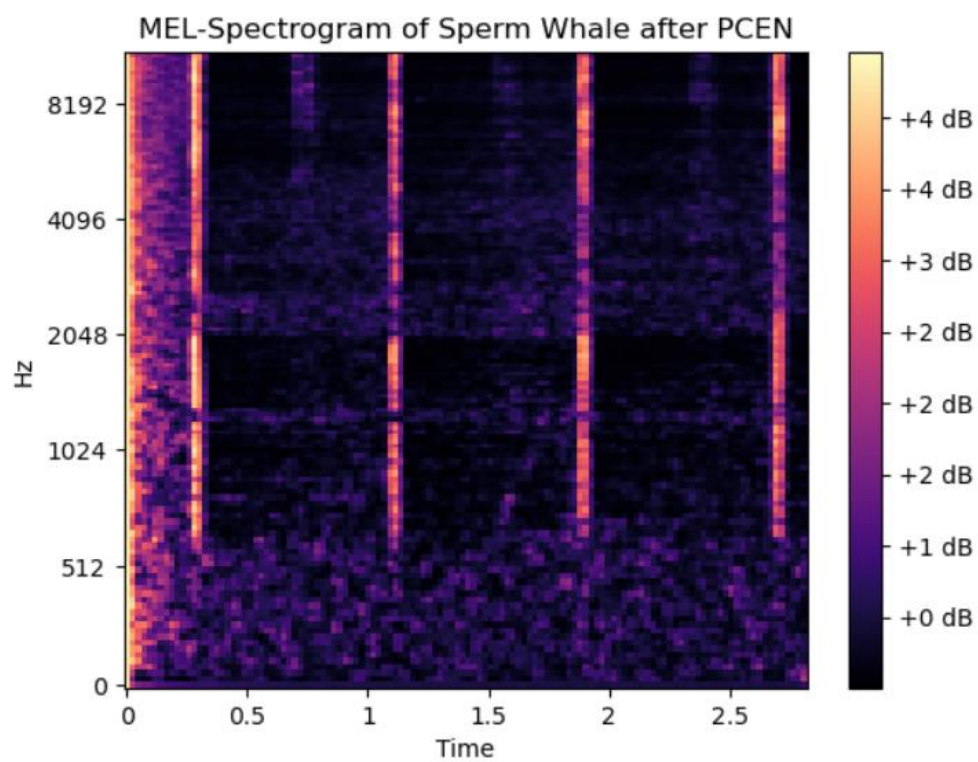
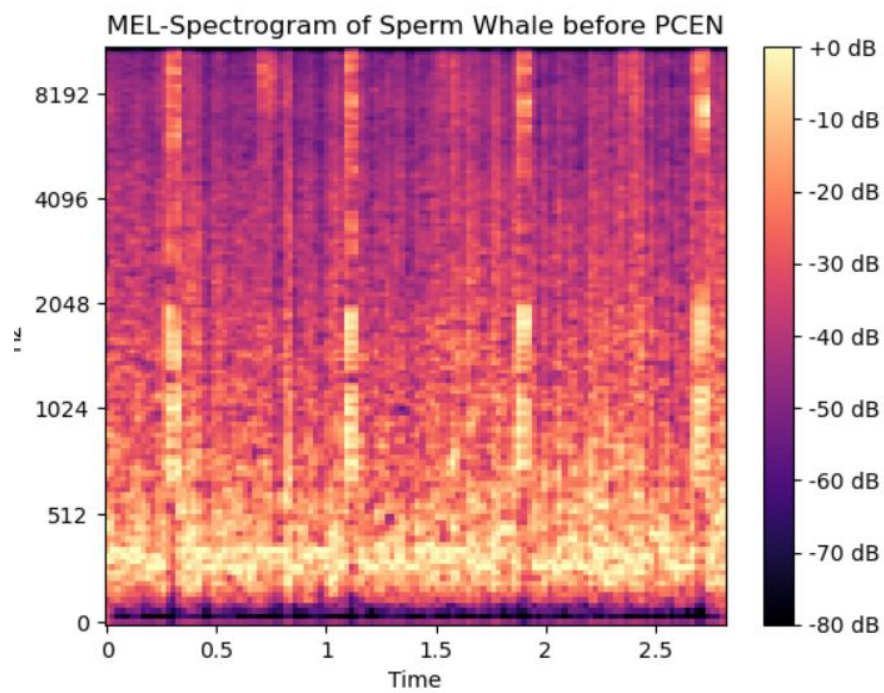
Frequency field: As mentioned in the previous question, we seem to *have attenuation of energy at higher energies* (going up the y-axis). Also, we seem to have energy in *a smaller frequency range in the humpback* whale (it does not reach the intense color as high as in the sperm) than in the sperm. Therefore, the spectrograms are *perfectly consistent* with the power density spectral diagrams.

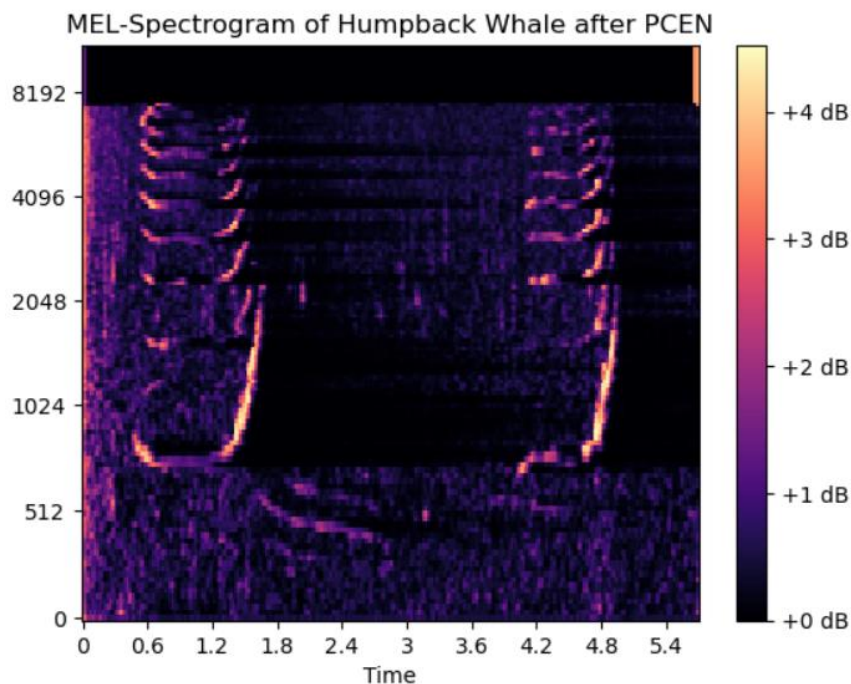
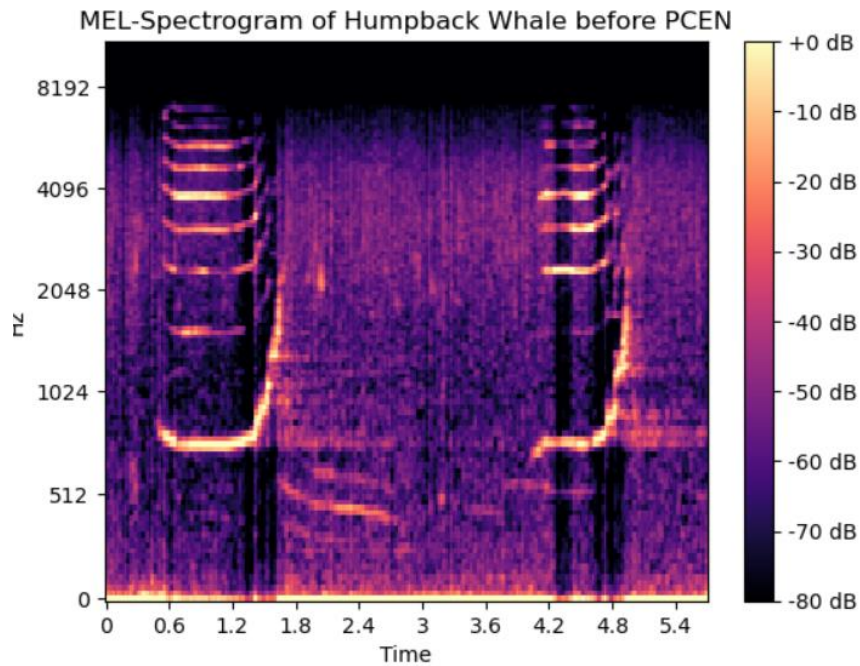
Question 3.10

In this question, we will construct with the help of the ***librosa.feature.melspectrogram()*** function the mel -spectrogram of the two biosignals before and after the application of the PCEN method (***librosa.pcen()***) which is a technique with which ***we enhance the contrast between noise and our signals.***

As for mel – spectrograms, *after searching the internet*, we concluded that ***they are spectrograms adapted to human acoustic frequencies*** that are not linear. Their main difference from the common spectrogram is the representation of frequencies on ***a mel scale***. These have applications in voice recognition.

Here are our results:





Regarding the distribution of energy in time and spectrum, we do not notice a difference from the previous conclusions we drawn , ***so spectrograms and power density spectral diagrams agree here as well.*** However, it is worth noting the ***strong contrast of colors*** after the PCEN method, which indicates that indeed ***the noise seems to have been***

limited by our graphs (***blacker color where there is noise and more intense color at signal points***).

*Obviously there is something wrong with the scales, as without normalization of the mel diagram we had a lot of decibels before the implementation of PCEN ***so we were forced to normalize***.